



**POLITÉCNICO
DE LEIRIA**

ESCOLA SUPERIOR
DE TECNOLOGIA
E GESTÃO

Plot In a Pot

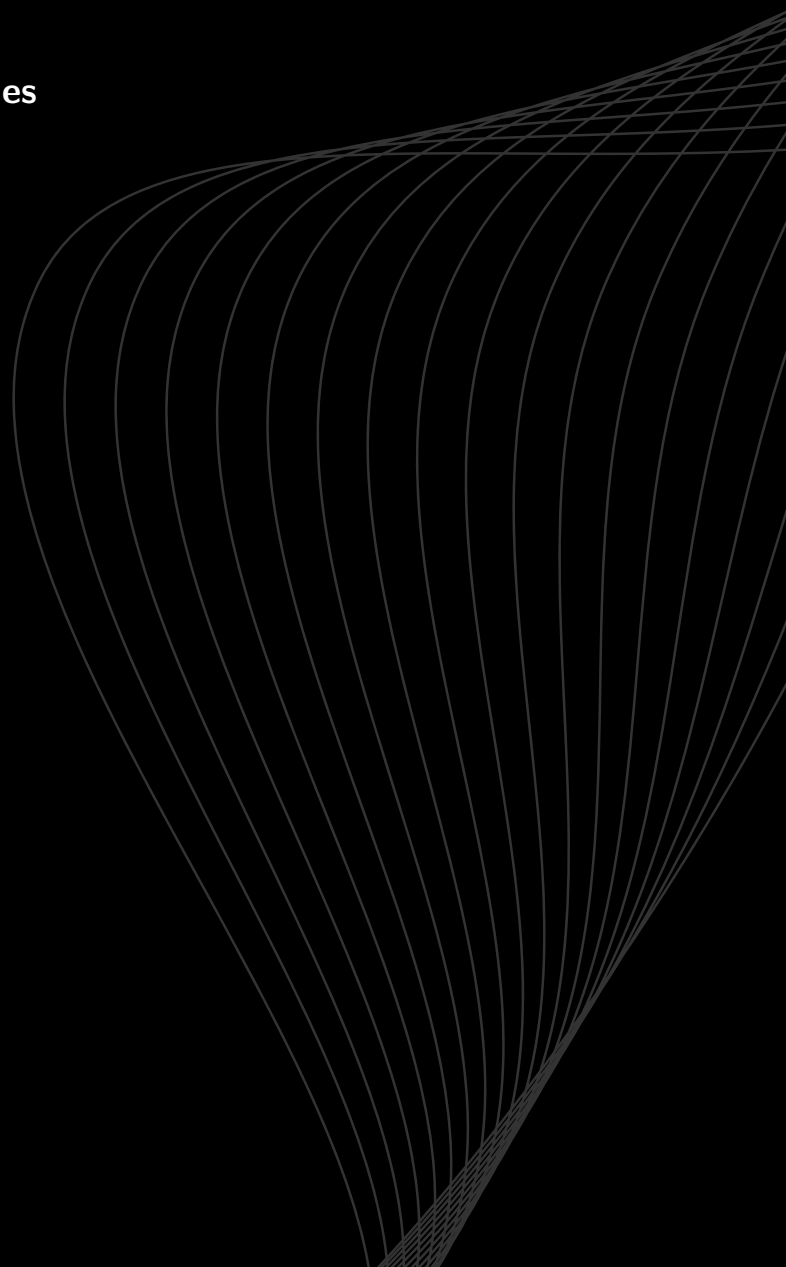
Uma Ferramenta para Narrativas Não Lineares

Miguel Gomes Silva

Tomás Alexandre dos Santos Reis Lopes

Escola Superior de Tecnologia e Gestão
Licenciatura em Engenharia Informática
UC de Projeto Informático

Leiria, Julho 2026



Plot In a Pot

Uma Ferramenta para Narrativas Não Lineares

Miguel Gomes Silva

Student No. 2221462

Tomás Alexandre dos Santos Reis Lopes

Student No. 2222970

Supervisores: Anabela Gonçalves Rodrigues Marto

*Professora Adjunta, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Miguel Cerdeira Marreiros Negrão

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Nelson Martins Ferreira

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Escola Superior de Tecnologia e Gestão

Departamento de Engenharia Informática

Licenciatura em Engenharia Informática

UC de Projeto Informático

Trabalho de Projeto da unidade curricular de Projeto Informático realizado sob a orientação da Professora Doutora Anabela Gonçalves Rodrigues Marto e do Professor Doutor Miguel Cerdeira Marreiros Negrão e do Professor Doutor Nelson Martins Ferreira

Leiria, Julho 2026

Plot In a Pot

Copyright © 2026 - Miguel Gomes Silva e Tomás Alexandre dos Santos Reis Lopes,
Escola Superior de Tecnologia e Gestão.

O presente relatório é um trabalho original, elaborado exclusivamente para este fim, tendo sido devidamente citados todos os autores cujos estudos contribuíram para a sua elaboração. É permitida a sua reprodução parcial com indicação dos autores e referência ao grau, ano letivo, instituição—*Politécnico de Leiria*—e data da defesa pública.

O presente trabalho beneficiou da utilização do modelo *IPLeiria-Thesis*.

Agradecimentos

Gostaríamos de expressar o nosso mais sincero agradecimento a todos aqueles que, direta ou indiretamente, contribuíram para a realização deste projeto informático e para a elaboração deste relatório.

Aos nossos orientadores, Professora Doutora Anabela Gonçalves Rodrigues Marto, Professor Doutor Nelson Martins Ferreira e Professor Doutor Miguel Cerdeira Marreiros Negrão, pelo voto de confiança, paciência e precioso apoio científico e técnico. O acompanhamento constante e a partilha do seu conhecimento foram essenciais para superar os desafios metodológicos deste trabalho. Agradecemos o rigor do Professor Nelson na abordagem matemática, a experiência do Professor Miguel na arquitetura de sistemas informáticos e a orientação estruturante da Professora Anabela, cujas discussões valiosas e revisões minuciosas permitiram elevar a qualidade técnica da nossa solução.

À Escola Superior de Tecnologia e Gestão (ESTG) do Politécnico de Leiria, por disponibilizar os recursos académicos e o ambiente propício ao nosso desenvolvimento pessoal e profissional ao longo da Licenciatura em Engenharia Informática.

Aos participantes que voluntariamente colaboraram nos testes de usabilidade e acessibilidade, cujo feedback construtivo e sincero foi inestimável para a validação e refinamento prático do software.

Por fim, expressamos um agradecimento muito especial às nossas famílias e amigos, pelo apoio incondicional, incentivo constante e paciência demonstrados durante este percurso académico.

Resumo

O desenvolvimento de narrativas interativas e jogos baseados em escolhas enfrenta sérios desafios de usabilidade e integridade estrutural à medida que a ramificação narrativa cresce. Embora existam ferramentas populares de autoria como o Twine, a maioria carece de suporte nativo integrado para gestão de localização multi-idioma e de motores de validação estática de caminhos lógicos para detetar inconsistências (como becos sem saída e cenas órfãs) antes da fase de compilação ou execução. O presente trabalho de projeto propõe o *Plot in a Pot* (PiaP), um editor visual e simulador baseado em grafos que unifica a autoria de narrativas escritas no formato aberto Tweak 3 (SugarCube) com um motor integrado de tradução de textos e validação estática de caminhos.

Implementado em React e no ecossistema React Flow, o PiaP destaca-se pela sua estética neo-brutalista de alto contraste visual, concebida para mitigar barreiras de acessibilidade visual e cromática para autores com daltonismo severo (p. ex. deuteranopia). O sistema integra: (i) uma matriz de localização baseada em grelha bidimensional com importação e exportação de CSV; (ii) um motor de validação estática suportado por algoritmos de Pesquisa em Largura (BFS) para analisar a acessibilidade de cenas; (iii) um modo de simulação interativo dotado de um agente inteligente autónomo (*Smart Walker*) que utiliza procura por novidade (*Novelty Search*) para percorrer e testar caminhos automaticamente; e (iv) a visualização de gráficos com arestas curvas para ligar as cenas.

A avaliação qualitativa e quantitativa do sistema foi efetuada através de testes com seis utilizadores com diferentes perfis técnicos e de acessibilidade. Os resultados revelaram uma taxa média de conclusão de tarefas de 95,8% e uma elevada usabilidade e satisfação, validando a eficácia do *Plot in a Pot* como uma ferramenta viável e robusta para autores e *game designers* na prototipagem e desenvolvimento de narrativas interativas complexas. Como trabalhos futuros, perspetiva-se a execução de IA local via WebGPU, edição colaborativa em tempo real (via modelos Ponto a ponto (Peer-to-peer) (P2P)) e model checking formal da lógica narrativa.

Palavras-Chave: Narrativa Interativa, Editores de Grafos, Localização de Jogos, Acessibilidade Visual, Validação Estática de Fluxo, Twine, SugarCube.

Abstract

The development of interactive narratives and choice-based games faces serious challenges in usability and structural integrity as the narrative branching expands. Although popular authoring tools like Twine exist, most lack integrated native support for multi-language localization management and static path-validation engines to detect inconsistencies (such as dead ends and orphan scenes) before compilation or execution. This project work proposes *Plot in a Pot* (PiaP), a graph-based visual editor and simulator that unifies the authoring of interactive stories written in the open Twee 3 format (SugarCube) with an integrated translation matrix and static path-validation engine.

Implemented in React and using the React Flow ecosystem, PiaP stands out for its high-contrast neo-brutalist visual style, specifically designed to mitigate visual and chromatic accessibility barriers for authors with severe color blindness (e.g., deuteranopia). The system features: (i) a translation matrix layout with CSV import and export capabilities; (ii) a static validation engine powered by Breadth-First Search (BFS) algorithms to audit scene reachability; (iii) an interactive simulation mode equipped with an autonomous agent (*Smart Walker*) that leverages novelty search heuristics to automatically test paths; and (iv) graph visualization with curved edges to connect scenes.

Qualitative and quantitative evaluation was conducted through user testing with six participants of varying technical backgrounds and accessibility needs. The results showed a 95.8% average task completion rate alongside high levels of usability and satisfaction, validating the effectiveness of *Plot in a Pot* as a viable and robust tool for authors and game designers during the prototyping and development of complex interactive narratives. Future work directions include local AI execution via WebGPU, real-time collaborative editing (via P2P models), and formal model checking of narrative logic.

Keywords: Interactive Narrative, Graph Editors, Game Localization, Visual Accessibility, Static Flow Validation, Twine, SugarCube.

Declaração sobre o Uso de Inteligência Artificial

Reconhecemos a utilização integrada do modelo de linguagem Gemini (<https://gemini.google.com>) como ferramenta de suporte no processo de desenvolvimento de software, refatoração de rotinas e automação de testes unitários para a aplicação *Plot in a Pot*. A inteligência artificial operou sob estrita validação e supervisão humana, funcionando exclusivamente como um assistente de produtividade para tarefas delimitadas de programação e validação de sintaxe.

Descrição da Utilização

Instrução 1: Gera uma expressão regular robusta em JavaScript para identificar a sintaxe de ligações Twee 3 do tipo `[[Texto|Destino]]` dentro de um bloco de texto corrido, e cria uma estrutura de testes unitários simples para validar se o parser em JavaScript captura corretamente o identificador do destino.

Resposta 1: [O modelo forneceu a expressão regular fundamentada para o motor do parser, acompanhada por um conjunto de asserções em Jest para testar variações na formatação de ligações com e sem espaços, o que serviu de base para a posterior implementação manual do ficheiro `tweeParser.js` na pasta de utilitários.]

Instrução 2: Como posso estruturar uma chamada HTTP nativa no React (utilizando a API Fetch) para enviar um texto de utilizador para a API local do Ollama (porta 11434) usando o modelo Llama 3.1 com streaming de dados desativado (`stream: false`)?

Resposta 2: [O assistente gerou a estrutura padrão da requisição POST contendo os cabeçalhos JSON necessários e o mapeamento básico do corpo do pedido para o endpoint `/api/generate`, que foi posteriormente integrada de forma segura na interface do utilizador da aplicação.]

Processo de Validação Humana e Decisão Prática:

Estima-se que cerca de 15 a 20% do código-base utilitário da aplicação tenha contado com o apoio da inteligência artificial para a geração de protótipos iniciais de algoritmos padrão. No entanto, todas as decisões críticas de engenharia de software e arquitetura permanecem sob inteira autoria e responsabilidade dos autores. O processo de validação,

correção e engenharia manual do sistema incluiu, de forma exclusiva:

- **Segurança e Soberania de Dados:** Toda a lógica de integração com o Ollama e tratamento de dados locais no cliente foi reconfigurada manualmente para garantir o cumprimento estrito do requisito RNF04, eliminando quaisquer fugas de informação para fora da máquina do utilizador.
- **Lógica Algorítmica Dinâmica:** Os algoritmos de validação estrutural baseados em pesquisas *BFS* (Breadth-First Search) e *DFS* (Depth-First Search) para detecção de ciclos, becos sem saída e nós órfãos na lista de adjacências bidirecional foram inteiramente projetados, escritos e depurados manualmente por nós.
- **Interoperabilidade Twee 3:** O parser síncrono e síncro-bidirecional foi meticulosamente revisto à mão para assegurar a fidelidade de conversão de coordenadas espaciais que o formato original do Twine não suporta nativamente.

A arquitetura final da aplicação, o fluxo de execução síncrona de dados no React Flow e o controlo de integridade semântica da história constituem propriedade intelectual exclusiva da autoria humana do projeto.

Conteúdo

<i>Lista de Figuras</i>	xv
<i>Lista de Tabelas</i>	xvii
<i>Siglas</i>	xix
1 Introdução	1
1.1 Objeto do Trabalho	1
1.2 Justificação e Pertinência do Tema	1
1.3 Objetivos do Trabalho	2
1.3.1 Objetivo Geral	2
1.3.2 Objetivos Específicos	2
1.4 Contributos do Trabalho	3
1.5 Estrutura do Documento	3
1.6 Estrutura do Documento	3
2 Fundamentação Teórica e Estado da Arte	4
2.1 Contexto Teórico	4
2.2 Conceitos Fundamentais de Narrativas Não-Lineares	5
2.3 Teoria de Grafos na Narrativa Interativa	6
2.3.1 Tipos de Grafos e Aplicação Narrativa	6
2.3.2 Modelação por Grafo Controlado por Estados	7
2.3.3 O Desafio da Explosão Combinatória	9
2.4 Software de Criação de Histórias não Lineares	9
2.4.1 Twine	9
2.4.2 Ink (Inkle Studios)	10
2.4.3 ChoiceScript	11
2.4.4 Yarn Spinner	11
2.4.5 Narrative Flow	12
2.4.6 Articy Draft	12
2.4.7 Análise Comparativa	13
2.5 O Formato Twee3 e Linguagens de Scripting Narrativo	14
2.5.1 Sintaxe de Ligações e Transições	14
2.5.2 Metadados Geométricos e Etiquetas (Tags)	15
2.5.3 Destinos Dinâmicos vs. Validação Estática	15

2.5.4	Macros e Lógica Condicional	16
2.6	Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais . .	16
2.6.1	Análise de Famílias de Modelos e Recomendações	16
2.6.2	Decisão de Design: Do Co-Criador ao Assistente de Conversão . .	17
3	Conceção e Desenvolvimento	19
3.1	Cronograma e Fases de Desenvolvimento	19
3.2	Arquitetura Geral do Sistema	20
3.3	Requisitos do Sistema	21
3.3.1	Requisitos Funcionais	21
3.3.2	Requisitos Não Funcionais	22
3.4	Métodos, Técnicas e Enquadramento Tecnológico	22
3.4.1	Justificação das Decisões de Design	23
3.5	Modelação de Dados: Lista de Adjacência	24
3.6	Parser Síncrono e Especificação Tweep 3	25
3.6.1	Importação e Conversão de Coordenadas de Zonas	25
3.6.2	Passagens Especiais	26
3.6.3	Modelo SugarCube (Ligações e Macros)	26
3.7	Interação Visual, Edição e Acessibilidade	27
3.7.1	Zonas de Agrupamento Visual	27
3.7.2	Arestas Curvadas com Waypoints	27
3.7.3	Acessibilidade e Identidade Visual Customizada	28
3.7.4	Modelação de Dados e Sincronização com o React Flow	29
3.8	Validação Estática e Simulação	31
3.8.1	Algoritmo de Validação Estática e Exploração do Espaço de Estados	31
3.8.2	Simulador em Tempo Real (PlayMode) e Smart Walker	34
3.8.3	Motor de Compilação JIT de Macros SugarCube	35
3.9	Integração de Modelos de Linguagem (LLMs)	36
3.9.1	Arquitetura de Integração Local	37
3.9.2	Papel do LLM: Conversão e Importação Automática de Histórias .	37
4	Implementação e Avaliação	39
4.1	Estrutura do Código e Detalhes de Implementação	39
4.1.1	Funcionamento da Interface Visual	39
4.1.2	Funcionamento da Camada de Lógica	41
4.2	Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)	43
4.2.1	Primeira Fase: Testes em Máquina Local (RTX 3050)	44
4.2.2	Segunda Fase: Testes no Servidor com GPU Titan Xp (12 GiB) . .	44
4.2.3	Correção de Caracteres Estranhos e Ajustes de Execução	44
4.2.4	Estado Atual da Funcionalidade (Beta)	45
4.3	Metodologia dos Testes de Utilizador	45
4.3.1	Perfil dos Participantes	45

4.3.2	Tarefas Avaliadas	46
4.4	Resultados dos Testes e Desenvolvimento Iterativo	46
4.4.1	Primeira Leva: Protótipo Inicial (Abril de 2026)	47
4.4.2	Segunda Leva: Protótipo Intermédio (Maio de 2026)	47
4.4.3	Terceira Leva: Protótipo Final (Junho de 2026)	48
4.4.4	Feedback Qualitativo e Usabilidade Visual	49
4.5	Discussão e Limitações do Estudo	49
4.5.1	Discussão Crítica do Desenvolvimento	50
4.5.2	Limitações Metodológicas da Avaliação	50
4.5.3	Limitações Tecnológicas e Funcionalidade de IA	50
5	Conclusão e Trabalho Futuro	52
5.1	Conclusão	52
5.1.1	Porquê Escolher o Plot in a Pot?	53
5.2	Trabalho Futuro	53
	<i>Bibliography</i>	57
	Apêndices	
A	Manual de Utilização e Instalação	63
A.1	Requisitos do Sistema e Instalação Local	63
A.1.1	Instalação do Editor Web	63
A.1.2	Configuração do Servidor de IA Local (Ollama)	64
A.2	Manual de Utilização do Editor Gráfico	64
A.2.1	O Canvas Interativo	64
A.2.2	Criação e Edição de Nós Narrativos (Cenas)	65
A.2.3	Criação de Ligações e Arestas de Escolha	65
A.2.4	Agrupamento em Zonas (Capítulos)	65
A.3	Programação de Lógica Narrativa e Variáveis	66
A.3.1	Gestão de Variáveis (Figma-Style Tokens)	66
A.3.2	Edição de Lógica condicional por Código	66
A.3.3	Editor de Lógica Estruturada	67
A.4	Tradução e Localização Multi-idioma	67
A.4.1	Configurar Idiomas	67
A.4.2	Editar Textos na Matriz	69
A.4.3	Importação e Exportação de Ficheiros Comma-Separated Values (CSV)	69
A.5	Validação Lógica e Simulação de Jogo	69
A.5.1	O Painel de Validação Estática	69
A.5.2	Simulação Ativa (PlayMode) e Testes com o Smart Walker	70
A.6	Operações de Ficheiros e Publicação do Projeto	70
A.6.1	Importar Ficheiros Twee 3	70

A.6.2	Exportar a História	70
B	Guião de Testes e Questionário de Avaliação	71
B.1	Guião de Tarefas de Teste	71
B.2	Questionário System Usability Scale (SUS)	72

Lista de Figuras

2.1	Diferentes tipologias de grafos e as suas representações em matrizes de adjacência.	8
2.2	Interface de utilizador do Twine com exibição gráfica de passagens e conexões.	10
2.3	Editor Inky para a linguagem Ink, com o painel de código (esquerda) e teste (direita).	11
2.4	Interface visual do Yarn Spinner integrada num ambiente de desenvolvimento.	12
2.5	Interface do editor visual do Narrative Flow com ligação de blocos lógicos. . .	12
2.6	Interface do Articy Draft X para planeamento profissional de histórias interativas.	13
3.1	Diagrama de blocos detalhado da arquitetura conceptual e fluxo de dados. . .	21
3.2	Exemplo de uma Zona de Agrupamento Visual com nós de cena filhos no canvas.	27
3.3	Exemplo de aresta suavizada com caminho curvado personalizável no canvas.	28
3.4	Interface gráfica global do editor (<i>Plot in a Pot</i>), ilustrando a identidade visual neo-brutalista de alto contraste com contornos espessos e sinalização redundante para acessibilidade.	29
4.1	Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).	40
4.2	Ecrã da sandbox de simulação (PlayMode), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.	41
4.3	Fluxograma do modelo de desenvolvimento iterativo centrado no utilizador e evolução SUS.	47
A.1	Diálogo de gestão de variáveis globais (VariablesModal), apresentando os tokens de dados e a renomeação segura baseada em Expressões Regulares (Regex).	67
A.2	Interface do editor visual de lógica estruturada (VisualBlocksEditor), dividindo as passagens em painéis de prosa, setters de variáveis e escolhas condicionais.	68
A.3	Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.	68

Lista de Tabelas

2.1	Comparação de características principais entre motores de narrativa interativa.	13
3.1	Cronograma detalhado das fases de desenvolvimento do projeto.	19
3.2	Mapeamento de operadores lógicos SugarCube para JavaScript.	37
4.1	Distribuição dos participantes pelas levas de testes de usabilidade.	46
4.2	Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).	48
4.3	Pontuação obtida no questionário System Usability Scale (SUS) na leva final.	49

Siglas

API	Application Programming Interface. <i>(p. 54)</i>
BFS	Pesquisa em Largura (Breadth-First Search). <i>(p. 7, 15, 19, 20, 24, 46, 48, 52, 71)</i>
CRDT	Tipos de Dados Replicados Sem Conflitos (Conflict-free Replicated Data Types). <i>(p. 54)</i>
CSV	Comma-Separated Values. <i>(p. xii, 2, 22, 46, 47, 48, 52, 69, 72)</i>
DFS	Pesquisa em Profundidade (Depth-First Search). <i>(p. 7, 20)</i>
JSON	JavaScript Object Notation. <i>(p. 15, 17, 21, 24, 26, 70)</i>
LLM	Modelo de Linguagem de Grande Escala (Large Language Model). <i>(p. 50, 54)</i>
NI	Narrativa Interativa. <i>(p. 1, 9, 13, 16, 50, 52)</i>
P2P	Ponto a ponto (Peer-to-peer). <i>(p. iii, v)</i>
PiaP	Plot in a Pot. <i>(p. 1, 2, 10, 15, 17, 21, 23, 24, 25, 29, 50, 52, 53, 63, 64, 66, 67)</i>
Regex	Expressões Regulares. <i>(p. xv, 67)</i>
SPA	Single Page Application. <i>(p. 23, 54)</i>
WebGPU	Web Graphics Processing Unit. <i>(p. 53, 54)</i>

1

Introdução

As narrativas interativas (Narrativa Interativa (NI)) colocam o utilizador no centro da tomada de decisões, permitindo-lhe moldar o rumo dos acontecimentos. Esta não-linearidade, contudo, aumenta exponencialmente a complexidade da escrita e do planeamento, exigindo ferramentas capazes de gerir fluxos de informação complexos e lógica de estados dinâmicos.

Neste contexto surge o (Plot in a Pot (PiaP)), um editor visual e simulador de narrativas interativas concebido para resolver os desafios de autores não-lineares, combinando uma interface visual baseada em grafos direcionados com um motor de simulação lógica em tempo real.

1.1 Objeto do Trabalho

O objeto deste projeto é o desenvolvimento do PiaP, um ambiente de desenvolvimento para criação, simulação e tradução de narrativas não-lineares. O sistema integra-se nativamente com o formato aberto Twee (versão 3) e com a macro-linguagem SugarCube do ecossistema Twine. Através de uma interface baseada em grafos, a ferramenta permite a autores e *narrative designers* gerir ramificações complexas e bases de dados de localização de forma unificada, garantindo compatibilidade com compiladores externos (como o Tweego) e controlo de versões em texto simples.

1.2 Justificação e Pertinência do Tema

O *design* de narrativas não-lineares enfrenta problemas severos de escala, onde árvores de diálogo e escolhas crescem de forma exponencial, tornando-se propensas a erros lógicos. Embora o Twine seja uma referência na prototipagem de NI (Interactive Fiction Technology Foundation, 2026), o desenvolvimento de projetos de grande escala é limitado pela falta de ferramentas nativas para a gestão de localização (multi-idioma) e pela ausência de validadores estáticos de fluxo que previnam caminhos sem saída antes da compilação.

O PiaP resolve esta lacuna ao introduzir uma estética neo-brutalista de alto contraste que otimiza o espaço de trabalho e melhora a acessibilidade de autores com limitações visuais (Babich, 2023; Loranger, 2022), permitindo isolar cadeias de texto rígidas através de chaves de tradução dinâmicas e oferecendo segurança matemática no mapeamento de rotas narrativas.

1.3 Objetivos do Trabalho

1.3.1 Objetivo Geral

Desenvolver uma plataforma *web* modular que unifique a edição visual de grafos de histórias interativas Twee com um motor integrado de localização multi-idioma e validação de caminhos lógicos.

1.3.2 Objetivos Específicos

Para a concretização do objetivo geral, definiram-se os seguintes objetivos específicos:

1. **Interface Gráfica Baseada em Grafos:** Desenvolver um *canvas* interativo de alto desempenho com estética neo-brutalista de alto contraste para a manipulação visual de nós, zonas, *scripts* em JavaScript e estilização em CSS.
2. **Encaminhamento Geométrico de Arestas:** Desenvolver um algoritmo de vetorização com *waypoints* interativos para as ligações entre nós, evitando a sobreposição visual de caminhos.
3. **Sincronização Bidirecional com Twee 3:** Construir um interpretador (*parser*) capaz de traduzir a sintaxe Twee 3 ([[Link|Destino]]) na topologia do grafo, gerindo as coordenadas espaciais.
4. **Módulo de Internacionalização Nativo:** Desenvolver uma matriz de localização para tradução direta por células, com suporte à importação e exportação em formato CSV.
5. **Análise Estática de Integridade Narrativa:** Implementar um motor de validação baseado em listas de adjacência para detetar automaticamente nós isolados, becos sem saída ou ciclos infinitos.
6. **Automação Narrativa por Inteligência Artificial:** Projetar uma arquitetura de inferência local (via LLM) para processar texto em linguagem natural e convertê-lo automaticamente na estrutura formal de grafos do motor.
7. **Ambiente de Execução e Aprendizagem:** Criar um simulador integrado (*PlayMode*) para testes em tempo real e um módulo pedagógico interativo (*Playable Tutorial*) para introdução assistida ao editor.

1.4 Contributos do Trabalho

Como resultado direto do projeto, foram alcançados os seguintes contributos práticos e científicos:

- **Plataforma Unificada de Autoria:** Criação de um ambiente que combina edição visual de grafos com a simulação reativa de variáveis de estado, unindo abordagens visuais e textuais.
- **Motor de Validação Estática Reativo:** Implementação de um validador baseado em grafos direcionados e Pesquisa em Largura (BFS) que audita a integridade narrativa em tempo real, alertando o autor sobre erros lógicos antes da compilação.
- **Inovação em Acessibilidade Prática:** Desenho de uma interface baseada em estética neo-brutalista de alto contraste e representação redundante de arestas por padrões de traço (contínuo/tracejado), validada como uma alternativa inclusiva para utilizadores daltónicos.
- **Centralização da Localização Narrativa:** Introdução de uma matriz de tradução por chaves abstratas com suporte para importação/exportação CSV (norma RFC 4180), eliminando textos fixos (*hardcoded*) nas passagens do Twine.
- **Inferência Local com Soberania de Dados:** Integração de um módulo de IA assente em LLMs de pesos abertos com execução local (via API Ollama), viabilizando a conversão rápida de texto natural para grafos sem partilha de dados externos.

1.5 Estrutura do Documento

1.6 Estrutura do Documento

Este relatório encontra-se estruturado em cinco capítulos principais. Além desta introdução, o Capítulo 2 (Fundamentação Teórica e Estado da Arte) apresenta uma análise comparativa das ferramentas existentes no mercado e descreve os conceitos matemáticos da teoria de grafos aplicados à narrativa. Seguidamente, o Capítulo 3 (Conceção e Desenvolvimento) detalha o planeamento do projeto, a arquitetura de software, a especificação Tweep 3, o modelo de listas de adjacências, os modos de visualização e a simulação lógica. Por sua vez, o Capítulo 4 (Implementação e Avaliação) descreve a estrutura física do código, os detalhes de implementação da aplicação e a metodologia e resultados dos testes de usabilidade realizados com utilizadores. Finalmente, o Capítulo 5 (Conclusão e Trabalho Futuro) sintetiza os resultados alcançados, as limitações identificadas e as propostas para desenvolvimentos futuros.

2

Fundamentação Teórica e Estado da Arte

Neste capítulo são expostos os fundamentos teóricos relativos à modelação matemática de fluxos narrativos interativos, bem como a análise do estado da arte sobre as ferramentas de autoria existentes, destacando os seus pontos fortes e limitações.

2.1 Contexto Teórico

A criação de narrativas não lineares enfrenta frequentemente a “Explosão de Complexidade”, na qual os modelos tradicionais baseados em sequências ou cadeias falham em produzir narrativas logicamente consistentes, resultando muitas vezes em conteúdo repetitivo ou contradições causais (Chen et al., 2021). A poluição visual nas ferramentas tradicionais surge porque fluxogramas simples ou cadeias de eventos não conseguem capturar de forma eficaz as ligações densas e multidimensionais entre batidas narrativas, conduzindo a problemas de “alucinação” em sistemas automatizados, nos quais os eventos previstos carecem de precisão ou de completude (Li et al., 2018). Sem uma estrutura formal, os designers enfrentam dificuldades significativas na gestão de percursos ramificados, o que pode conduzir a evoluções narrativas incoerentes que são cognitivamente exigentes de depurar ou verificar manualmente (Mormoris, 2024).

Para ultrapassar estas limitações estruturais, *frameworks* recentes recorrem a *Narrative Event Evolutionary Graphs* (NEEG) e a *Event-centric Temporal Knowledge Graphs*, de forma a fornecer uma representação estruturada da evolução dos eventos (Li et al., 2018; Yan et al., 2023). Ao tratar as histórias como grafos direcionados com relações temporais e causais explícitas, estes sistemas conseguem garantir um fluxo lógico e prevenir as inconsistências encontradas em designs manuais tipo “esparguete” (Chen et al., 2021; Li et al., 2018; Tang et al., 2022). A implementação de uma *framework* deste tipo permite a utilização de conjuntos de exclusão mútua e métricas de coerência para assegurar que os eventos simultâneos são lógicos e que cada batida narrativa permanece alcançável dentro da hierarquia pretendida (Chen et al., 2021). Esta mudança para uma

organização baseada em grafos transforma as ferramentas narrativas de meras aplicações de desenho em validadores lógicos ativos, garantindo consistência global através das propriedades matemáticas formalizadas da interação entre eventos (Chen et al., 2021; Tang et al., 2022).

Tendo em conta o facto de ainda não existirem normas estabelecidas, ao formalizar uma narrativa como um Conjunto Parcialmente Ordenado (Poset)¹, este projeto pode ir além da simples visualização para criar um Validador Lógico que assegura a integridade estrutural ao nível de arquitetura.

2.2 Conceitos Fundamentais de Narrativas Não-Lineares

Antes de abordar a modelação computacional, importa clarificar a evolução e os conceitos essenciais que regem as narrativas não-lineares. Ao contrário do modelo tradicional, cuja progressão textual é puramente sequencial do início ao fim, uma **narrativa não-linear** caracteriza-se pela fragmentação do discurso e pela existência de múltiplos percursos de leitura.

Do ponto de vista estrutural, e independentemente da tecnologia utilizada, qualquer história ramificada assenta em três elementos fundamentais:

- **Cenas (Nós):** Os fragmentos autónomos de conteúdo. sejam estes textuais, visuais ou sonoros. Contêm a substância dramática exposta ao leitor num determinado momento da história.
- **Escolhas (Ligações):** Os mecanismos de bifurcação expostos ao longo da narrativa que convidam o utilizador a realizar uma escolha, determinando o vetor de transição para o fragmento de conteúdo seguinte.
- **Memória Narrativa (Estado):** O conjunto de dados acumulados que regista o histórico de ações, a posse de objetos ou os parâmetros quantificáveis do percurso do leitor. Esta memória altera o contexto da obra em tempo real, ditando se certos caminhos se tornam acessíveis ou bloqueados no futuro.

Para ilustrar estas dinâmicas de forma clara, considere-se o seguinte exemplo narrativo:

A sua personagem acorda numa clareira envolta por uma floresta densa e percebe que acabou de amanhecer. Ao olhar em redor com mais detalhe, nota dois elementos que lhe chamam a atenção: uma casa abandonada por entre as árvores e um trilho no lado oposto. A casa parece trancada, mas pressente que conseguirá abrir a porta se encontrar algo para forçar a entrada. O trilho, por outro lado, aparenta não oferecer qualquer tipo de bloqueio.

Para onde vai a sua personagem?

- Casa abandonada

¹ As propriedades de reticulado são fundamentais para garantir a integridade lógica da narrativa.

- Trilho oposto

Neste cenário, a descrição inicial da clareira constitui a **Cena** ativa (o nó de partida). As opções dadas ao utilizador representam as **Escolhas** (as ligações ou arestas do grafo) que despoletam a transição estrutural. Se o utilizador optar por seguir o trilho e, mais à frente, recolher uma ferramenta, essa ação é registada na **Memória Narrativa** (o estado). Numa interação futura, caso decida regressar à casa abandonada, o motor avaliará retroativamente essa memória para validar se o caminho para o interior da habitação se deve abrir ou permanecer trancado.

2.3 Teoria de Grafos na Narrativa Interativa

A estrutura básica de uma história interativa assenta na modelação geométrica e lógica de caminhos narrativos. Um fluxo de história pode ser formalmente representado como um Grafo $G = (V, E)$ (Bondy et al., 2008), onde:

- V representa o conjunto de **Vértices** (ou nós), que correspondem a cenas, passagens de texto ou estados lógicos da história.
- E representa o conjunto de **Arestas** (ou ligações direcionadas), que representam as escolhas disponibilizadas ao leitor para transitar de uma cena para a outra.

De seguida, são apresentadas as diferentes estruturas de grafos aplicadas ao design narrativo, a comparação fundamental entre fluxos acíclicos e com ciclos, e a fundamentação matemática que justifica a necessidade de ferramentas automatizadas para mitigar a explosão de complexidade lógica.

2.3.1 Tipos de Grafos e Aplicação Narrativa

Dependendo da natureza do jogo ou da história, utilizam-se diferentes tipos de estruturas de grafos (ilustrados na Figura 2.1):

1. **Grafo Direcionado (Directed Graph):** Uma estrutura onde as ligações têm direção, ou seja, a existência de um caminho de $A \rightarrow B$ não implica o retorno de $B \rightarrow A$. Numa matriz de adjacência (uma tabela bidimensional que mapeia as conexões entre todos os nós do grafo, onde a linha i e a coluna j indicam a existência de uma ligação do nó i ao nó j), isto traduz-se numa estrutura não-simétrica:

$$M[i][j] \neq M[j][i]$$

Representa o fluxo narrativo puro, onde cada escolha conduz o leitor numa direção específica.

2. **Grafo Não-Direcionado (Undirected Graph):** Representa relações simétricas e bidirecionais ($A - B$). Embora seja menos comum em narrativa linear, é útil para modelar redes de relacionamento social entre personagens ou conexões espaciais

de livre exploração (p. ex., salas adjacentes de uma masmorra). Na matriz de adjacência, a estrutura é simétrica:

$$M[i][j] = M[j][i]$$

3. **Grafo de Conhecimento (Knowledge Graph):** Modela relações semânticas complexas entre entidades (p. ex., “personagem conhece objeto”, “porta depende de chave”). Exige múltiplas matrizes de adjacência ou uma estrutura tridimensional:

$$M_{relation}[i][j]$$

É ideal para jogos com inventários complexos e decisões baseadas em conhecimento indireto acumulado pelo utilizador.

4. **Grafo Ponderado (Weighted Graph):** Atribui valores (pesos) às arestas:

$$M[i][j] = \text{peso}$$

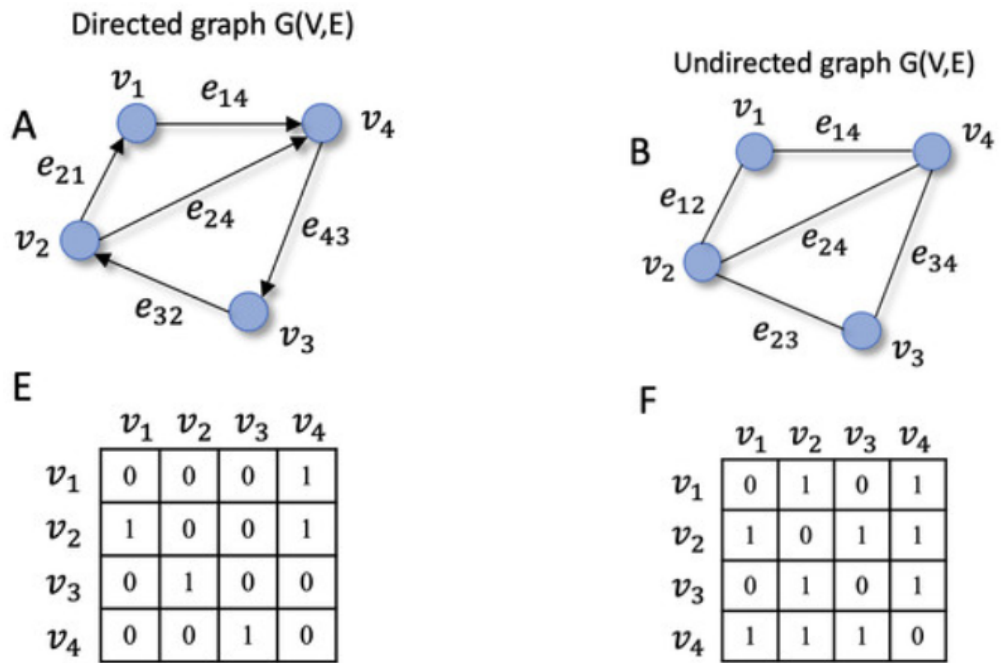
Estes pesos podem representar custos de recursos (tempo, pontos de vida), custos emocionais do personagem, ou a probabilidade de uma escolha ter sucesso.

2.3.2 Modelação por Grafo Controlado por Estados

No desenvolvimento de sistemas narrativos, é frequente restringir a estrutura a Grafos Direcionados Acíclicos (*DAGs*) para garantir um fluxo estritamente unidirecional. Contudo, essa abordagem impede a criação de mecânicas essenciais como *hubs* centrais de exploração ou ciclos de passagem de tempo baseados em ações recorrentes.

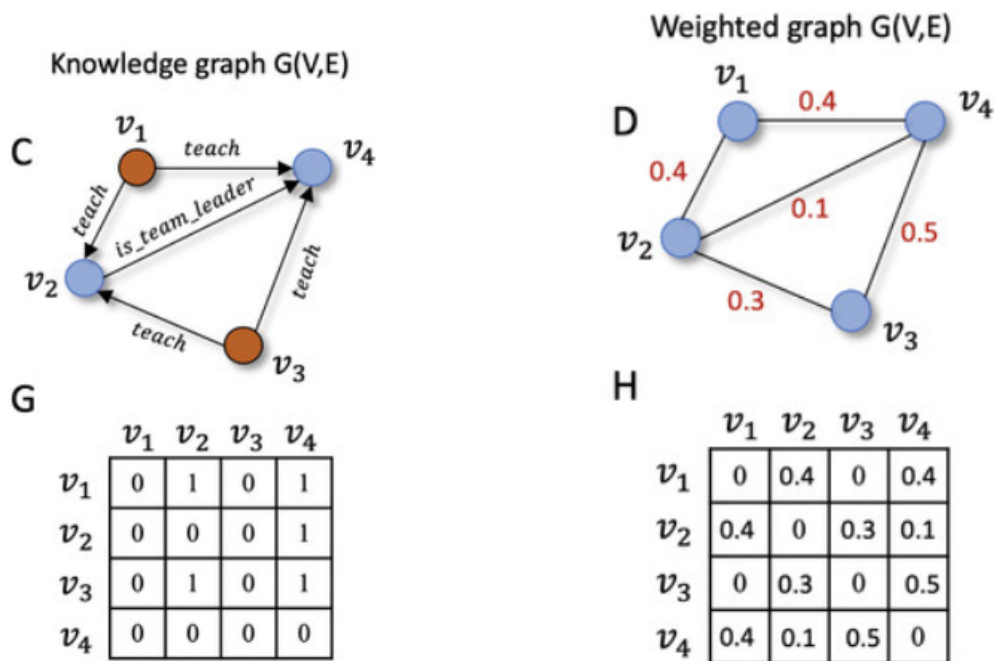
Por este motivo, o *Plot in a Pot* admite a existência de ciclos, mas afasta-se do modelo convencional de grafo direcionado simples onde uma aresta une rigidamente um nó *A* a um nó *B*. Em vez disso, a aplicação implementa um modelo de **Grafo Controlado por Estados**. No contexto da ferramenta, as ligações entre as cenas não são estáticas; o caminho efetivamente percorrido depende diretamente das macros condicionais (p. ex. `<<if>>`) e do estado atual das variáveis do utilizador.

Isto significa que a estrutura real da história não é determinada apenas pelo visual do *canvas*, mas sim pela computação em tempo de execução. Uma única escolha desenhada no editor pode direcionar o jogador para múltiplos destinos alternativos, dependendo estritamente do valor das variáveis em memória. Esta flexibilidade exige algoritmos de validação em segundo plano (como pesquisas Pesquisa em Largura (Breadth-First Search) (BFS) e Pesquisa em Profundidade (Depth-First Search) (DFS) (Cormen et al., 2009)) adaptados para analisar a conectividade considerando o impacto destas variáveis antes da compilação final.



(a) Grafo Direcionado (A) e respetiva Matriz de Adjacência (E).

(b) Grafo Não-Direcionado (B) e respetiva Matriz de Adjacência (F).



(c) Grafo de Conhecimento (C) e respetiva Matriz de Adjacência (G).

(d) Grafo Ponderado (D) e respetiva Matriz de Adjacência (H).

Figura 2.1: Diferentes tipologias de grafos e as suas representações em matrizes de adjacência.

2.3.3 O Desafio da Explosão Combinatória

A necessidade de validação automatizada no desenvolvimento de narrativas interativas prende-se com o fenómeno da **explosão combinatória**. À medida que o autor adiciona novas escolhas e bifurcações à história, o número de caminhos lógicos possíveis cresce de forma exponencial.

Este problema agrava-se com a introdução de ciclos (que permitem regressar a cenas anteriores) e variáveis de estado (como inventários ou escolhas morais). Em projetos de grande dimensão, a complexidade resultante torna impossível a um autor humano auditar manualmente todas as combinações de escolhas e validar se existem caminhos bloqueados, becos sem saída ou loops infinitos onde o utilizador possa ficar preso. Justifica-se, assim, o desenvolvimento do motor de verificação estática do *Plot in a Pot*, capaz de explorar automaticamente todas as possibilidades lógicas do projeto e alertar o criador sobre anomalias na escrita em tempo real.

2.4 Software de Criação de Histórias não Lineares

O conceito de NI refere-se a narrativas estruturadas onde a progressão do texto não é linear, dependendo diretamente das escolhas e ações submetidas pelo utilizador para navegar caminhos alternativos (Montfort, 2005). Embora a tecnologia digital tenha popularizado o termo, a literatura não-linear física remonta a precursores como os livros-jogo (*gamebooks*) e a série *Choose Your Own Adventure* popularizada nas décadas de 1970 e 1980 (Packer, 1984), bem como a experiências literárias ramificadas pioneiras (p. ex., o conto de Jorge Luis Borges em 1941) (Borges, 1941; Aarseth, 1997). Com a expansão recente do mercado de videojogos independentes (Juul, 2005; Adams, 2014) e o desenvolvimento da narratologia computacional (Tang et al., 2022; Yan et al., 2023), emergiram várias ferramentas de autoria gráfica e linguagens de scripting para suportar a criação de estruturas ramificadas (Interactive Fiction Technology Foundation, 2026).

Apresenta-se, de seguida, uma análise detalhada das principais referências do mercado académico e profissional.

2.4.1 Twine

Lançado em 2009 por Chris Klimas (Klimas, 2009), o Twine é a ferramenta mais popular na comunidade de NI baseada em texto.

- **Vantagens:** Extremamente acessível (sem barreira de programação para escrita básica); visualização clara baseada em nós gráficos no canvas; open-source e gratuito (distribuído sob a licença GPL v3, permitindo qualquer utilização, incluindo comercial, sem restrições); rápido para prototipagem.
- **Limitações:** A interface visual torna-se desorganizada e lenta em projetos muito grandes (efeito “prato de esparguete”); a lógica de estado nativa é limitada; a integração com motores de jogos externos (como Unity ou Unreal) é complexa.

O fluxo de trabalho central do Twine assenta na criação de passagens (segmentos individuais da história) e na sua ligação através de hiperligações que representam escolhas do jogador. Estas passagens são exibidas como caixas num canvas com setas a indicar os caminhos narrativos (Klimas, 2009), conforme ilustrado na Figura 2.2. Este paradigma mapeia-se diretamente para a representação em canvas de grafos interativos que constitui o núcleo visual do PiaP, tornando a correspondência entre o modelo de dados interno e a interface imediata e semanticamente coerente.

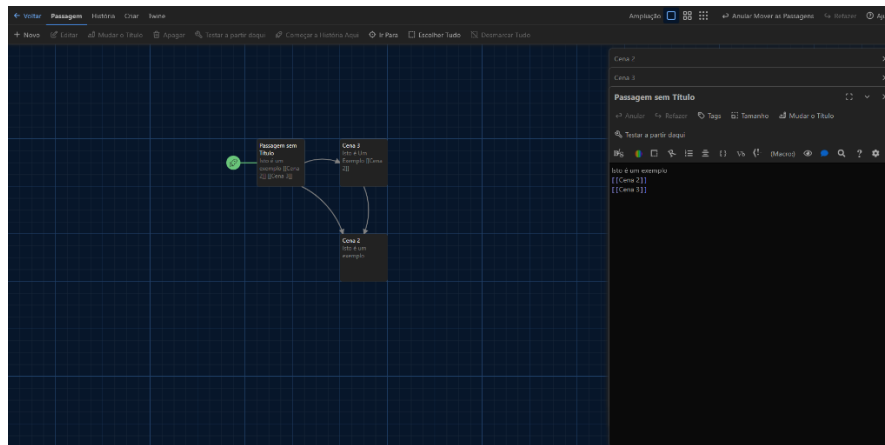


Figura 2.2: Interface de utilizador do Twine com exibição gráfica de passagens e conexões.

2.4.2 Ink (Inkle Studios)

O Ink (Inkle Studios, 2016) é uma linguagem textual de scripting criada pela Inkle Studios (criadores de jogos como *80 Days*). A sua escrita é efetuada no editor Inky (apresentado na Figura 2.3), que oferece pré-visualização em tempo real da narrativa criada.

- **Vantagens:** Altamente poderosa para o controlo de estados narrativos através de variáveis locais e globais, condicionais que ativam escolhas dinamicamente e gestão de ramificações complexas assente em sub-nós lógicos (*stitches* e *gathers*); excelente integração técnica com motores de jogo (tais como Unity e Unreal Engine, que contam com suporte oficial da editora); licenciada sob a licença MIT (totalmente livre); escala muito bem para projetos de grande envergadura.
- **Limitações:** Não é uma ferramenta visual, operando estritamente através de ficheiros de texto estruturados. A ausência de um canvas ou representação espacial de grafos de nós dificulta o planeamento geográfico da narrativa, tornando impossível visualizar de forma imediata o fluxo geral ou detetar de forma intuitiva incoerências, ciclos infinitos e becos sem saída sem uma análise exaustiva e manual de milhares de linhas de código por parte de autores não-técnicos; exige fortes conhecimentos de lógica de programação.

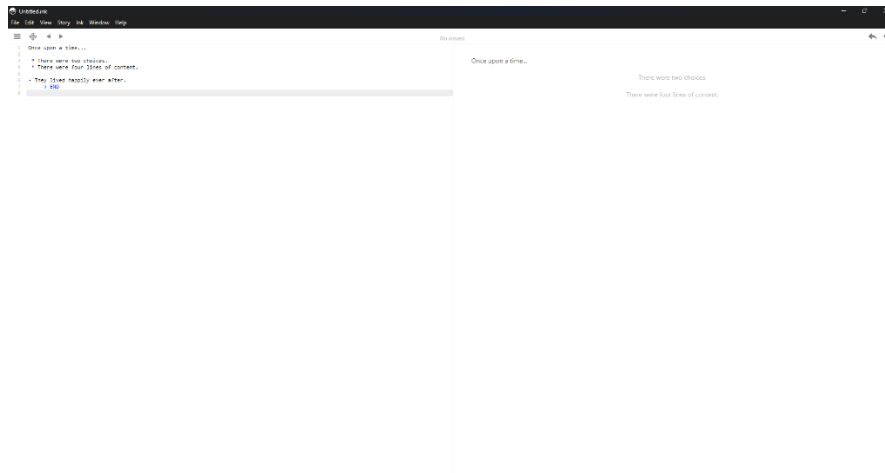


Figura 2.3: Editor Inky para a linguagem Ink, com o painel de código (esquerda) e teste (direita).

2.4.3 ChoiceScript

Desenvolvido pela Choice of Games, o ChoiceScript é uma linguagem de programação simples e baseada em texto para a escrita de romances interativos baseados em escolhas.

- **Vantagens:** Sintaxe simples e linear de fácil compreensão para escritores; possui excelentes ferramentas de teste automático (como `quicktest` e `randomtest`) que facilitam a validação de caminhos e a robustez lógica de romances longos.
- **Limitações:** Não é uma ferramenta visual; o formato de ficheiro é proprietário; a sua licença (CSL) proíbe explicitamente a utilização comercial sem autorização prévia e acordo de distribuição comercial com a Choice of Games (Choice of Games, 2026), limitando a autonomia de autores independentes.

2.4.4 Yarn Spinner

O Yarn Spinner (Secret Lab, 2015) é uma ferramenta focada na integração de diálogos em jogos interativos com motor gráfico (ilustrada na Figura 2.4).

- **Vantagens:** Sintaxe simples (semelhante a scripts de teatro); suporte nativo para variáveis; excelente portabilidade direta para Unity e Unreal Engine.
- **Limitações:** Carece de um planeador visual abstrato robusto fora do motor de jogo principal; não é adequado para desenhar fluxogramas abstratos de histórias antes da fase de produção do jogo.

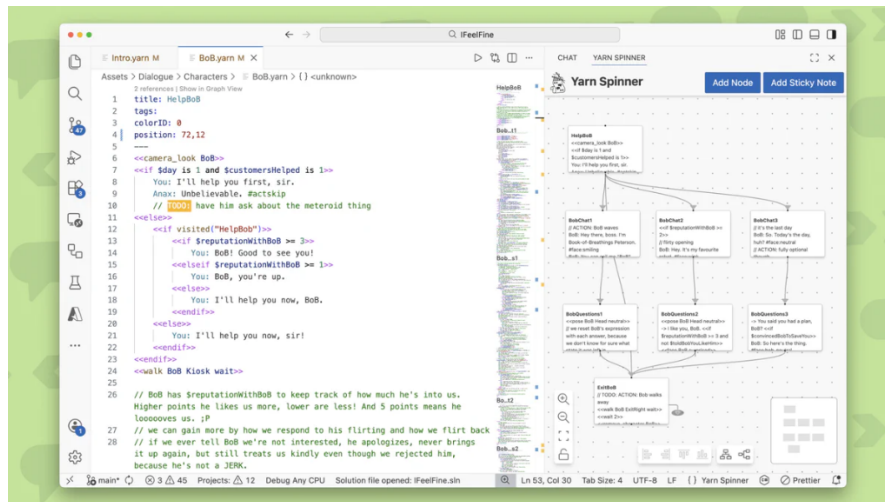


Figura 2.4: Interface visual do Yarn Spinner integrada num ambiente de desenvolvimento.

2.4.5 Narrative Flow

O Narrative Flow (NarrativeFlow, 2023) é uma ferramenta emergente que tenta colmatar a lacuna entre Twine e Ink através de um editor visual baseado em cartões (Figura 2.5).

- **Vantagens:** Oferece melhor organização estrutural do que o Twine e uma visualização mais clara que o Ink; ajuda a gerir a consistência da escrita.
- **Limitações:** É um software pago, com menor maturidade técnica e uma comunidade de utilizadores consideravelmente mais reduzida.

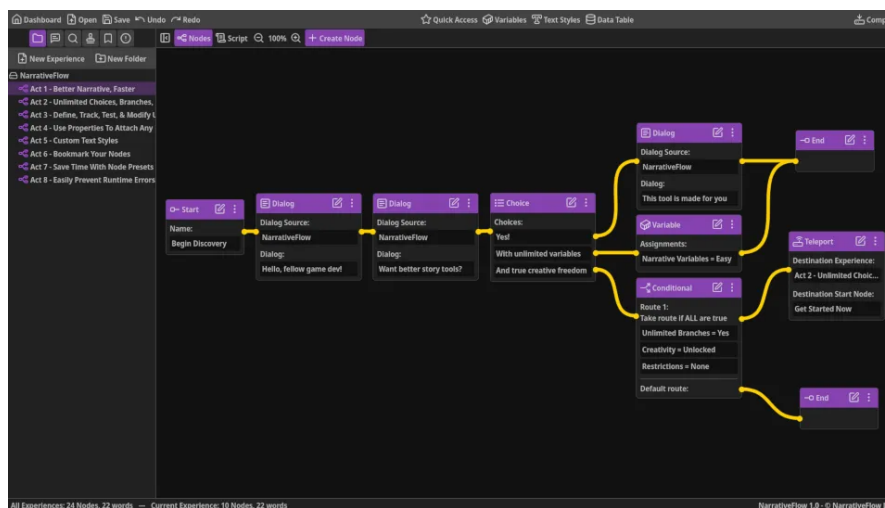


Figura 2.5: Interface do editor visual do Narrative Flow com ligação de blocos lógicos.

2.4.6 Articy Draft

O Articy Draft (Articy Software, 2010) é a solução comercial líder da indústria de videojogos AAA para planeamento de narrativas e bases de dados de escrita (apresentado na Figura 2.6).

- **Vantagens:** Altamente estruturado e profissional; gestão integrada de personagens, inventários e diálogos; escala com eficácia para equipas de desenvolvimento de grandes dimensões.
- **Limitações:** Curva de aprendizagem acentuada; software dispendioso (licença comercial paga); excessivamente complexo e pesado para autores independentes ou projetos pequenos.

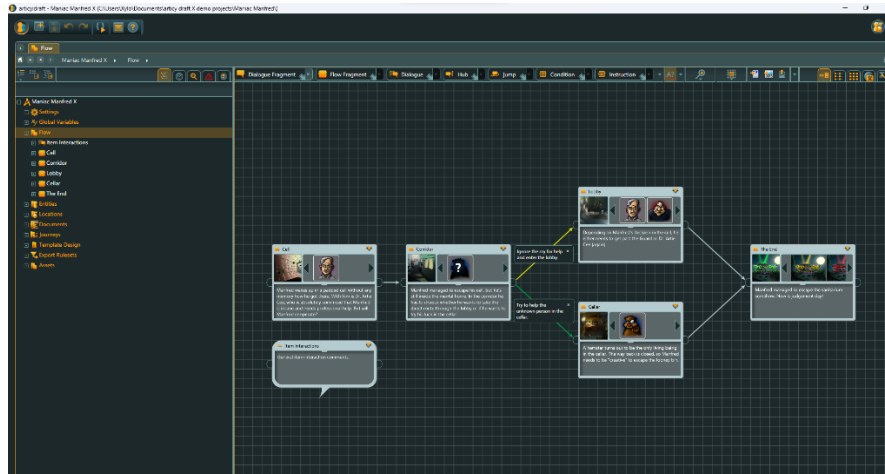


Figura 2.6: Interface do Articy Draft X para planeamento profissional de histórias interativas.

2.4.7 Análise Comparativa

A fim de estruturar a comparação entre as diferentes abordagens, apresenta-se na Tabela 2.1 uma síntese comparativa das ferramentas baseada nos seus recursos, suporte a lógica de jogo e preços.

Tabela 2.1: Comparação de características principais entre motores de narrativa interativa.

Característica	Twine	Ink	Choice-Script	Yarn Spinner	Narrative Flow	Articy Draft
Edição Visual	Sim	Não	Não	Sim	Sim	Sim
Variáveis e Estado	Sim	Sim	Sim	Sim	Sim	Sim
Lógica Condicional	Sim	Sim	Sim	Sim	Sim	Sim
Verificação Estática	Básica	Básica	Não	Não	Não	Não
Integração (Engines)	Não	Sim	Não	Sim	Sim	Sim
Gratuito e Aberto	Sim	Sim	Sim	Sim	Não	Não

A seleção dos critérios de avaliação apresentados na Tabela 2.1 justifica-se pela necessidade de analisar de que forma as soluções existentes respondem aos desafios práticos da autoria e produção de NI:

- **Edição Visual:** Identifica se o motor oferece uma representação gráfica em forma de canvas ou nós para visualização e mapeamento direto dos caminhos narrativos (essencial para autores não-técnicos).

- **Variáveis e Lógica Condicional:** Avalia o suporte para manter uma memória narrativa (estado) e a capacidade de restringir escolhas dinamicamente com base nas ações do leitor.
- **Verificação Estática:** Determina se a ferramenta realiza auditorias automáticas para detetar becos sem saída, nós inacessíveis ou incoerências lógicas em tempo de desenho. Embora motores como o *Ink* realizem verificação de integridade de desvios (*diverts*) inválidos durante a compilação, e o editor do *Twine* sinalize visualmente ligações quebradas (sendo ambas assinaladas como "Básica"), estas soluções carecem de uma análise estática formal capaz de modelar e verificar de forma contínua e interativa o espaço de estados de variáveis e acessibilidade lógica geral em tempo de desenho.
- **Integração com Engines:** Mede a portabilidade da narrativa para motores gráficos como Unity, Unreal Engine ou Godot, indispensável para produções de jogos modernos.
- **Gratuito e Aberto:** Reflete a acessibilidade financeira e a ausência de restrições comerciais ou licenciamento fechado na distribuição das obras criadas.

2.5 O Formato Twee3 e Linguagens de Scripting Narrativo

Para além da representação visual em grafos, o ecossistema de narrativa interativa apoia-se em formatos de serialização de texto que permitem aos autores escrever e armazenar as suas obras em ficheiros de texto simples (.txt ou .twee). O padrão da indústria para esta representação é o **Twee3**, uma especificação aberta mantida pela *Interactive Fiction Technology Foundation* (IFTF) (Interactive Fiction Technology Foundation, 2026).

O formato Twee3 organiza o ficheiro narrativo através de blocos lógicos denominados **Passagens** (Passages). Cada passagem começa com um identificador de início (duas frações de dois pontos seguidas do título da passagem):

Listing 2.1: Estrutura básica de uma passagem em Twee3.

```

1  :: Floresta
2  Acordas numa floresta densa e silenciosa.
3  Ves um trilho a tua frente.
4
5  [[Seguir pelo trilho->Trilho]]

```

2.5.1 Sintaxe de Ligações e Transições

No Twee3, as bifurcações narrativas e as escolhas do leitor são representadas por parênteses retos duplos. Existem duas sintaxes principais para definir transições:

- **Ligação Direta:** [[Nome da Passagem]], onde o texto da escolha apresentado ao leitor é idêntico ao título da passagem de destino.

- **Ligação Rotulada:** `[[Texto da Escolha->Destino]]`, onde o texto visível é separado do identificador da passagem por uma seta.

2.5.2 Metadados Geométricos e Etiquetas (Tags)

Para além do conteúdo narrativo e das ligações, a especificação normativa do Twee3 suporta a inclusão de metadados em formato JavaScript Object Notation (JSON) e de etiquetas (*tags*) diretamente no cabeçalho das passagens. O cabeçalho estendido obedece à seguinte estrutura:

Listing 2.2: *Exemplo de cabeçalho estendido com etiquetas e metadados geométricos em Twee3.*

```
1 :: Nome da Passagem [etiqueta1 etiqueta2]
   {"position":"100,200","size":"100,100"}
```

Onde:

- **Etiquetas (Tags):** Declaradas entre parênteses retos (`[...]`) após o título da passagem, servem para catalogar ou aplicar propriedades especiais de visualização ou lógica.
- **Metadados JSON:** Delimitados por chavetas (`{...}`), contêm dados estruturados de persistência da história. As propriedades padrão para a representação visual em editores gráficos são a `position` (coordenadas *X,Y* absolutas no ecrã) e a `size` (largura e altura físicas da caixa no canvas).

2.5.3 Destinos Dinâmicos vs. Validação Estática

A especificação normativa do Twee3 permite que os autores utilizem expressões lógicas ou variáveis dinâmicas como destinos de transição (por exemplo, `[[Seguir em frente|$destino]]` ou `[[Voltar|Trilho + $passagemAnterior]]`). Em tempo de execução, o motor de jogo resolve o valor da variável e executa o redirecionamento para o nó correspondente.

Embora esta flexibilidade dinâmica traga vantagens de modularidade, ela impede a representação estática do fluxo narrativo sob a forma de um grafo determinista e inviabiliza análises automáticas de integridade antes da execução. Visto que o destino de um nó não pode ser determinado sem correr o jogo passo a passo, o compilador fica impossibilitado de construir a lista de adjacências e mapear as arestas visuais.

Por esta razão, o PiaP introduz uma restrição sintática ao formato Twee3: todos os destinos de ligação devem ser estáticos e literais (isto é, referenciar textualmente o nome de uma passagem existente no grafo). Ao abdicar desta funcionalidade dinâmica de transição, a plataforma ganha a capacidade de:

- Desenhar de forma determinista as arestas direcionadas que ligam as passagens no canvas interativo.
- Executar com precisão o algoritmo de validação estática de caminhos baseada em BFS para detetar nós órfãos e becos sem saída lógicos.

- Viabilizar a exploração de estados e simulação inteligente de caminhos efetuada pelo agente *Smart Walker*.

2.5.4 Macros e Lógica Condicional

A especificação Twee3 é independente do formato de exibição (story format) utilizado em tempo de execução (como Harlowe, SugarCube ou Chapbook). Cada um destes formatos fornece um conjunto de **Macros** para manipulação do estado da narrativa:

- **Declaração de Estado:** Permite criar ou modificar variáveis na memória narrativa. Por exemplo, em SugarCube: `<<set $temChave = true>>`.
- **Estruturas Condicionais:** Permitem ocultar ou exibir escolhas e caminhos com base no estado atual das variáveis. Por exemplo:

Listing 2.3: *Exemplo de logica condicional em Twee3.*

```

1 <<if $temChave>>
2     [[Abrir a porta trancada->InteriorCasa]]
3 <<else>>
4     A porta esta trancada e precisas de algo para a forçar.
5 <</if>>

```

Esta estrutura híbrida que combina geometria de grafos, marcação de texto simples e scripting condicional é a especificação que o *Plot in a Pot* adota nativamente, permitindo importar ficheiros externos de outras ferramentas (como o Twine) e exportar o trabalho final num formato de texto aberto e interoperável.

2.6 Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais

Com o avanço recente no campo da Inteligência Artificial Generativa, a integração de Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs) locais e gratuitos foi analisada numa pesquisa realizada a 13 de abril. O objetivo inicial desta pesquisa foi avaliar a viabilidade prática de utilizar estas ferramentas para auxiliar autores na escrita criativa, na sugestão de caminhos e na co-criação de narrativas interativas, analisando o potencial de diferentes famílias de modelos para tarefas de geração e estruturação textual.

2.6.1 Análise de Famílias de Modelos e Recomendações

Nessa pesquisa realizada a 13 de abril, foram analisadas três das principais famílias de modelos em open-weight ajustados para instruções, avaliando a sua capacidade e adequabilidade para autoria de NI:

1. **Família Llama 3 e 3.1 (Meta AI):**

- **Versão Recomendada:** Llama 3.1 (8B).
- **Justificação:** Representa um dos modelos leves mais capazes do estado da arte. A versão de 8 mil milhões de parâmetros corre com excelente desempenho na maioria dos computadores modernos equipados com placa gráfica dedicada ou em arquiteturas Mac Apple Silicon (M1/M2/M3), requerendo investimentos de hardware reduzidos.
- **Vantagem Narrativa:** Demonstra uma capacidade excecional no cumprimento estrito de diretrizes de formatação (*system prompts* rígidos). Esta fiabilidade torna-o ideal para gerar respostas estruturadas (por exemplo, em formato JSON contendo a sugestão de cena e as escolhas de saída), facilitando a integração direta no motor lógico do software.

2. Família Mistral (Mistral AI):

- **Versão Recomendada:** Mistral NeMo (12B) ou Mistral v0.3 (7B).
- **Justificação:** Os modelos desenvolvidos pela empresa europeia Mistral AI são amplamente reconhecidos pela comunidade de escrita criativa pelo seu tom mais orgânico e natural. Adicionalmente, tendem a apresentar menor nível de censura prévia, facilitando a escrita livre em géneros literários como ficção geral, mistério ou terror.
- **Vantagem Narrativa:** Apresentam janelas de contexto extremamente eficientes. A janela de contexto atua como a memória de curto prazo do modelo; numa narrativa ramificada, é fundamental que o LLM recorde o histórico de cenas e as decisões anteriores tomadas pelo utilizador para garantir a coerência semântica global do texto gerado.

3. Gemma 2 (Google):

- **Versão Recomendada:** Gemma 2 (9B).
- **Justificação:** Apesar do seu tamanho compacto de 9 mil milhões de parâmetros, destaca-se pelo seu desempenho elevado em raciocínio abstrato e análises textuais comparativas.
- **Vantagem Narrativa:** É altamente eficaz na realização de resumos e sínteses. Esta capacidade é ideal para funcionalidades auxiliares que leiam um determinado ramo completo da história e gerem um resumo conciso (p. ex., “descreve os principais acontecimentos deste arco narrativo”).

2.6.2 Decisão de Design: Do Co-Criador ao Assistente de Conversão

Embora a pesquisa inicial (Secção 2.6) tenha demonstrado o potencial dos LLMs para sugerir cenas e resumir narrativas, o desenvolvimento do PiaP tomou a decisão de design de não incluir funcionalidades de geração automática ou co-criação literária na aplicação final. Esta escolha foi motivada pelo respeito à autoria humana exclusiva e pela forte oposição da comunidade de escritores a conteúdos gerados de forma não-humana, priorizando a integridade do processo de escrita do autor.

Desta forma, o papel do LLM foi deliberadamente limitado a uma função técnica e utilitária: um assistente de importação rápida que converte texto livre (já escrito pelo autor) para a sintaxe estruturada Tweepi interpretada pelo parser.

Para viabilizar esta aplicação sem custos de API ou quebras de privacidade, a integração é feita localmente com ferramentas como o *Ollama* ou o *LM Studio*, que disponibilizam o modelo numa porta local (11434) para chamadas HTTP internas. A integração segue uma divisão de trabalho estrita para garantir a fiabilidade do sistema:

- **Lógica Estrutural (Grafo):** A validação de caminhos, a gestão do histórico e as regras de jogo são processadas de forma 100% determinística pelo código da aplicação (Lista de Adjacência Bidirecional). Isto garante que a estrutura da narrativa nunca é corrompida.
- **Assistente de Importação (LLM):** Atua apenas na análise semântica do texto ao nível do vértice (cena), convertendo o texto do autor para a marcação do parser ou sugerindo conexões textuais, sem interferir na integridade do grafo global.

3

Conceção e Desenvolvimento

Este capítulo descreve o cronograma e as fases de desenvolvimento do projeto, a arquitetura geral do *Plot in a Pot*, detalhando as decisões de design de dados, o funcionamento dos algoritmos de processamento, as estratégias de visualização e acessibilidade adotadas, e o motor de simulação implementado.

3.1 Cronograma e Fases de Desenvolvimento

O projeto foi executado ao longo de um período aproximado de três meses, utilizando uma abordagem de desenvolvimento incremental assente em prototipagem rápida. O cronograma de implementação dividiu-se em cinco fases fundamentais, estruturadas conforme se descreve de seguida:

Tabela 3.1: *Cronograma detalhado das fases de desenvolvimento do projeto.*

Fase	Descrição das Atividades Realizadas	Intervalo de Datas
Fase 1	Levantamento de requisitos e desenho conceitual da arquitetura.	17/03/2026 – 31/03/2026
Fase 2	Programação do núcleo (<i>core</i>) lógico: parser Twee 3 e validador BFS.	01/04/2026 – 25/04/2026
Fase 3	Interface e visualização: integração do React Flow e <i>waypoints</i> nas arestas.	26/04/2026 – 20/05/2026
Fase 4	Integração de IA local (Ollama) e otimização de acessibilidade visual.	21/05/2026 – 10/06/2026
Fase 5	Testes com utilizadores, correção de erros e refinamento final.	11/06/2026 – 25/06/2026

Fase 1 – Desenho e Requisitos: Definição das especificações funcionais e não funcionais do sistema. Elaboração do diagrama de arquitetura modular para separar a camada gráfica do processamento de grafos.

Fase 2 – Motor Lógico: Implementação da biblioteca `tweeParser.js` para ler e estruturar o formato Twee 3 em memória como uma lista de adjacências bidirecional,

e do motor de validação estática assente em travessias BFS.

Fase 3 – Componentes de Canvas: Acoplamento do React Flow e desenho de nós customizados, incluindo a lógica de grupos (Zonas) com gestão de pointer events, e as arestas editáveis baseadas em curvas Spline Catmull-Rom.

Fase 4 – IA e Acessibilidade: Configuração de chamadas HTTP para o Ollama local e validação da conversão de texto livre em nós Tweep. Ajustes no design de Tailwind CSS para implementar a estética neo-brutalista de alto contraste e a redundância de traços para daltónicos.

Fase 5 – Validação Prática: Realização de testes estruturados com utilizadores finais para identificar bloqueios cognitivos na interface e falhas de usabilidade.

3.2 Arquitetura Geral do Sistema

O sistema foi desenhado segundo uma arquitetura modular dividida em três camadas conceptuais independentes de tecnologia. Esta abordagem garante a separação de responsabilidades entre a recolha de comandos, o processamento de dados e a sua persistência, promovendo a manutenibilidade e a evolução isolada de cada módulo:

1. **Camada de Apresentação (Interface):** Responsável por gerir a interação direta com o utilizador. É constituída por:
 - **Canvas de Grafos:** O espaço visual interativo onde o autor cria, manipula e liga os nós narrativos.
 - **Matriz de Tradução:** Uma grelha estruturada que expõe os dicionários de localização e permite a tradução de chaves de texto em tempo real.
 - **Ecrã do Simulador:** A interface interativa de jogo onde o autor testa e experimenta a narrativa sob o ponto de vista do leitor.
2. **Camada de Processamento Lógico (Núcleo):** Contém a inteligência do sistema e os motores de transformação. É constituída por:
 - **Parser Tweep 3:** Módulo encarregue de ler ficheiros de texto estruturados em formato Tweep 3 e convertê-los no modelo interno da aplicação, bem como serializar e exportá-los de volta.
 - **Validador Estático:** Motor que executa travessias de grafo (BFS/DFS) para inspecionar inconsistências estruturais, como ciclos sem saída, nós inacessíveis ou hiperligações quebradas.
 - **Motor de Simulação:** Sandbox responsável por inicializar scripts customizados, interpretar condicionais lógicas e controlar o estado de jogo na simulação. Este estado é representado por um conjunto de variáveis lógicas globais e locais que registam as decisões do utilizador (ex: inventário, escolhas de caminhos ou pontuações).
3. **Camada de Dados e Persistência:** Representa o modelo de informação e o armazenamento. É constituída por:

- **Lista de Adjacência:** O modelo de dados em memória que representa a topologia do grafo, as saídas (sucessores) e entradas (predecessores) de cada cena.
- **Dicionários JSON:** A base de dados estruturada que armazena os dicionários de tradução e localização no armazenamento local.

O fluxo de funcionamento da aplicação e a relação de comunicação entre estas camadas encontram-se representados no diagrama detalhado da Figura 3.1.

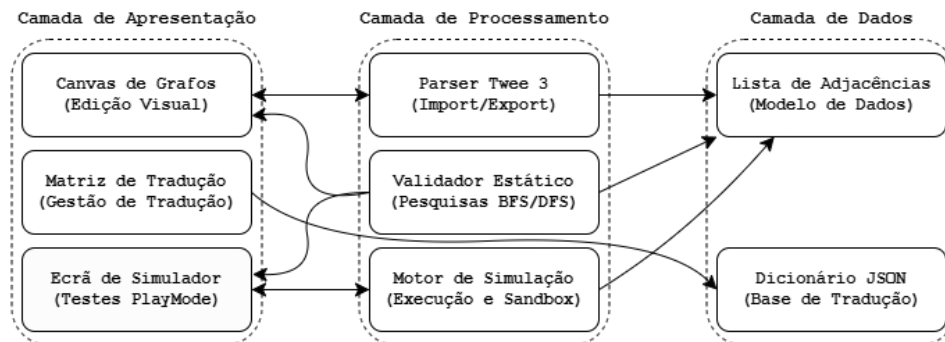


Figura 3.1: Diagrama de blocos detalhado da arquitetura conceptual e fluxo de dados.

3.3 Requisitos do Sistema

Com o intuito de delimitar o âmbito do projeto, o sistema foi estruturado para responder a necessidades fundamentais de modelação visual, internacionalização nativa, análise estática de integridade e inteligência artificial no cliente. Para a materialização prática do PiaP, estas necessidades foram detalhadas sob a forma de requisitos técnicos, divididos entre funcionais (comportamento direto da aplicação) e não funcionais (restrições de qualidade, desempenho e usabilidade).

3.3.1 Requisitos Funcionais

- RF01 - Criação e Edição de Nós Narrativos (Cenas):** O utilizador deve poder instanciar, arrastar, atualizar (título e corpo de texto das cenas) e remover de forma atómica nós narrativos no canvas bidimensional.
- RF02 - Ligações Direcionadas entre Nós (Escolhas):** O sistema deve inferir e instanciar arestas direcionadas automaticamente no grafo a partir da análise reativa de hiperligações double-bracket (`[[Texto|Destino]]`) inseridas pelo autor no texto das passagens.
- RF03 - Parser e Compatibilidade Twee 3:** O sistema deve assegurar a interoperabilidade bidirecional síncrona com a especificação Twee 3, suportando a importação e exportação de ficheiros com extensão `.twee`, mantendo a sintaxe e efetuando o mapeamento geométrico das coordenadas espaciais dos nós.

- RF04 - Auditoria e Validação Estática de Fluxo:** A plataforma deve executar análises estáticas e reativas sobre a topologia do grafo em plano secundário, sinalizando inconsistências estruturais como nós órfãos (inacessíveis), becos sem saída (hiperligações para identificadores inexistentes) ou caminhos inacessíveis devido a restrições lógicas.
- RF05 - Matriz de Localização Multi-idioma:** O sistema deve disponibilizar uma matriz de tradução tabular centralizada que associe chaves dinâmicas das passagens a múltiplos dicionários de tradução, permitindo a exportação e importação síncrona de ficheiros em formato CSV em conformidade com a norma RFC 4180.
- RF06 - Simulador em Tempo Real (PlayMode):** A aplicação deve integrar um simulador interativo em tempo real (*PlayMode*) executado sob uma sandbox isolada do DOM, interpretando macros SugarCube, avaliando condições lógicas e executando scripts ou estilos customizados.
- RF07 - Zonas de Agrupamento Visual:** O utilizador deve ser capaz de delimitar contêineres espaciais redimensionáveis (Zonas) para agrupamento lógico e organização de nós de cena, permitindo a sua translação conjunta no canvas.
- RF08 - Importação Inteligente via Conversor LLM:** O sistema deve disponibilizar um assistente de inteligência artificial capaz de converter narrativas lineares em linguagem natural para o formato estruturado Tweep 3, recorrendo à integração assíncrona com modelos de linguagem locais (Ollama) ou APIs externas.

3.3.2 Requisitos Não Funcionais

- RNF01 - Portabilidade e Execução no Cliente (Browser):** A plataforma deve ser executável inteiramente no lado do cliente através de um navegador de internet padrão, sem requerer servidores ou infraestruturas complexas.
- RNF02 - Usabilidade e Acessibilidade Visual:** A interface de grafos deve utilizar a estética neo-brutalista e garantir que a visualização de fluxos (arestas de entrada e saída) seja distinguível sem dependência exclusiva de cor, apoiando utilizadores daltónicos.
- RNF03 - Desempenho e Fluidez:** O canvas do React Flow e a conversão do parser síncrono devem suportar dezenas de nós em simultâneo com latência de interação e atualização abaixo de 100 milissegundos.
- RNF04 - Privacidade e Soberania de Dados:** Todos os dados introduzidos, incluindo ficheiros importados, tabelas de tradução e dados submetidos para inteligência artificial no Ollama local, devem permanecer estritamente no armazenamento local do utilizador para garantir segurança absoluta.

3.4 Métodos, Técnicas e Enquadramento Tecnológico

Para concretizar as especificações da arquitetura conceptual e responder aos requisitos definidos, o projeto apoia-se em metodologias ágeis de prototipagem rápida e numa

seleção de tecnologias executadas do lado do cliente. As principais opções metodológicas e técnicas encontram-se estruturadas a seguir:

- **Desenvolvimento da SPA (React 19) e Estilização (Tailwind CSS 3):** A aplicação foi inteiramente estruturada e programada pela equipa como uma Single Page Application (SPA) em **React 19** (Meta Open Source, 2013), tirando partido da nossa experiência prévia com esta tecnologia para gerir reativamente a árvore de componentes da interface. A identidade visual neo-brutalista (de alto contraste e bordas sólidas) foi desenhada e implementada por nós através da aplicação de classes utilitárias de **Tailwind CSS 3** (Tailwind Labs, 2017) nos componentes do editor.
- **Extensões ao Canvas de Grafos (ReactFlow 11):** Em vez de desenvolvermos um motor de desenho vetorial do zero, utilizámos a biblioteca **ReactFlow 11** (webkid.io, 2021) como base para gerir interações de zoom e arrasto. O nosso trabalho consistiu em estender esta infraestrutura básica através do desenvolvimento de nós totalmente customizados de cena (`StoryNode.jsx`) e de agrupamento (`ZoneNode.jsx`), além de arestas interativas personalizadas.
- **Algoritmia de Validação (Teoria de Grafos):** Concebemos e implementámos um modelo de dados baseado em listas de adjacências bidirecionais locais. Sobre esta estrutura, programámos algoritmos específicos para realizar análises topológicas em plano secundário (detecção de inconsistências via BFS) e a simulação de caminhos heurística do *Smart Walker*.
- **Sistema de Internacionalização (i18next):** Integrámos a biblioteca **i18next** para gerir dicionários locais. Desenvolvemos o mapeamento que associa os dados tabulares da matriz de tradução do utilizador a ficheiros JSON de localização estáticos, carregados dinamicamente no navegador com o plugin de cliente `i18next-http-backend` sob o modelo *backendless*.

3.4.1 Justificação das Decisões de Design

Com base na análise comparativa das ferramentas existentes (ver Secção 2.4.7), as escolhas de design e de tecnologia para o PiaP foram fundamentadas nos seguintes pontos:

- **Fusão de Paradigmas (Visual vs. Textual):** Enquanto o Twine se destaca pela interface puramente visual e o Ink pelo controlo fino de estados lógicos complexos, o PiaP visa fundir ambas as forças, proporcionando um editor visual de grafos que não abdica da manipulação robusta de condicionais e variáveis (via macro-linguagem SugarCube).
- **Adoção do Formato Aberto Twee 3 e Limitação de Grafos:** A especificação aberta Tweep 3 possui um mapeamento direto entre passagens e grafos, o que suporta a decisão de adoptá-lo como base de persistência do PiaP. Contudo, para permitir que o sistema leia as saídas e as renderize visualmente como arestas de forma fiável, o projeto introduz um compromisso de design (*trade-off*) em relação

ao Twine tradicional: não é suportada a utilização de variáveis como destinos dinâmicos de hiperligações (ex: `[[Ir para o local|$local]]`), exigindo que todos os destinos do grafo sejam estáticos (conforme detalhado na Secção 2.5.3).

- **Validação Automática e Segurança do Fluxo:** O ChoiceScript é valorizado pelas suas ferramentas robustas de teste automático (`randomtest`), uma lacuna crítica do Twine. O PiaP incorpora de raiz este princípio ao disponibilizar o validador estático (BFS) e o Smart Walker no simulador para preencher esta falta, garantindo a correção imediata do fluxo narrativo.
- **Autonomia e Licenciamento Livre:** A restrição de uso comercial imposta pelo ChoiceScript salienta a importância de desenvolver uma ferramenta que promova a autonomia dos autores, permitindo a exportação de projetos sob licenças livres e sem taxas adicionais.

3.5 Modelação de Dados: Lista de Adjacência

Para representar o fluxo da história no editor, o sistema utiliza uma **Lista de Adjacência Bidirecional**. Cada cena no modelo de dados armazena de forma independente dois conjuntos de caminhos:

- **Saídas (Sucessores):** Um vetor que contém as escolhas explícitas disponíveis para o leitor, mapeando o texto da opção e o ID do nó de destino.
- **Entradas (Predecessores):** Um vetor que regista as referências de todos os outros nós que contém escolhas a apontar para o nó atual.

Esta representação permite à aplicação realizar o mapeamento reverso instantâneo (*backward mapping*), alimentando os painéis “Vens de...” e “Vais para...” na interface. A estrutura de dados JSON representativa de um nó apresenta o seguinte formato simplificado:

```

1 {
2   "id": "cena_praca",
3   "titulo": "A Praça Central",
4   "conteudo": "O sol brilha no mercado central.",
5   "saidas": [
6     { "texto_escolha": "Entrar na taberna", "destino": "cena_taberna" }
7   ],
8   "entradas": ["cena_taberna"]
9 }
```

Note-se que este modelo foca-se exclusivamente na estrutura topológica do grafo. As variáveis globais e as condições lógicas SugarCube não são serializadas na lista de adjacências, sendo integradas diretamente no corpo de texto (`conteudo`) das passagens e interpretadas em tempo de execução pelo simulador.

3.6 Parser Síncrono e Especificação Twee 3

Para além do processamento interno em memória, a plataforma exige um mecanismo de serialização para guardar e carregar as histórias de forma estruturada. Em vez de criar um formato de ficheiro proprietário (como um esquema JSON customizado), optou-se por adotar a especificação aberta **Twee 3** como o padrão de persistência de dados da aplicação.

Esta decisão fundamenta-se em três vantagens fundamentais para o ecossistema do projeto:

- **Interoperabilidade e Compatibilidade Bidirecional:** A adoção do Twee 3 garante que qualquer história criada no PiaP possa ser exportada e editada diretamente no Twine oficial (ou noutros editores compatíveis), e vice-versa. Isto evita o aprisionamento tecnológico (*vendor lock-in*) e permite a integração em fluxos de trabalho já existentes.
- **Compilação e Execução via Tweego:** O formato Twee 3 é a especificação nativa interpretada pelo compilador de linha de comandos Tweego. Isto possibilita que os ficheiros gerados pelo nosso editor sejam compilados de forma direta em ficheiros HTML executáveis com todos os recursos e estilos do formato SugarCube.
- **Legibilidade e Controlo de Versões:** Sendo um formato baseado em texto simples, os ficheiros `.twee` são facilmente legíveis por humanos e ideais para controlo de versões (utilizando sistemas como o Git), permitindo acompanhar as alterações literárias e lógicas de forma clara ao longo do tempo.

O módulo responsável por fazer a ponte entre o modelo visual e a representação textual é o parser síncrono, implementado no ficheiro `tweeParser.js` (localizado em `src/utils/tweeParser.js`). Este módulo trata da sincronização bidirecional em tempo real entre o canvas do React Flow e o formato de texto Twee 3 (Interactive Fiction Technology Foundation, 2020).

Esta sincronização acontece de forma síncrona diretamente na memória do navegador. Isto significa que qualquer alteração de coordenadas, edição de texto ou criação de novas ligações no canvas é traduzida de imediato para a estrutura Twee 3 interna (e o oposto também se verifica), sem tempos de espera ou passos de compilação. Como a ferramenta funciona inteiramente no lado do cliente (*client-side*), este mapeamento apenas mantém o estado da sessão atualizado; a persistência física dos dados no disco rígido requer sempre que o utilizador exporte o ficheiro de forma explícita.

3.6.1 Importação e Conversão de Coordenadas de Zonas

Durante a leitura e escrita de ficheiros, o parser converte as coordenadas de posição absolutas guardadas no formato Twee 3 (explicadas na Secção 2.5.2) em coordenadas locais quando um nó é filho de uma Zona (explicado na Secção 3.7.1). O React Flow exige que os nós filhos tenham posições relativas ao canto superior esquerdo da zona

mãe, calculadas por:

$$X_{\text{relativo}} = X_{\text{absoluto}} - X_{\text{zona}}$$

$$Y_{\text{relativo}} = Y_{\text{absoluto}} - Y_{\text{zona}}$$

No processo inverso de exportação para ficheiro `.twee`, o parser executa a conversão inversa para manter a compatibilidade com outros compiladores de Twine:

$$X_{\text{absoluto}} = X_{\text{zona}} + X_{\text{relativo}}$$

$$Y_{\text{absoluto}} = Y_{\text{zona}} + Y_{\text{relativo}}$$

3.6.2 Passagens Especiais

O parser reconhece passagens reservadas de metadados da especificação Tweep 3:

- **:: StoryTitle:** Define o título do projeto.
- **:: StoryData:** Contém as configurações globais em formato JSON, incluindo o identificador único da história (`ifid`), o formato de compilação (`SugarCube`), o nó de arranque (`start`) e o nível de zoom.
- **:: StoryInit:** Nó especial não-narrativo onde são declaradas e inicializadas as variáveis lógicas de controlo do estado da simulação (as quais servem de memória do jogo, conforme introduzido na Secção 3.2).

3.6.3 Modelo SugarCube (Ligações e Macros)

As arestas são extraídas do texto narrativo de cada nó através de expressões regulares que identificam ligações no formato clássico Tweep:

- **Arestas Simples:** `[[Cena2]]`
- **Arestas com Alias ou Direcionamento:** `[[Ir para a Taverna|cena_taberna]]` ou `[[Abrir porta->cena_interior]]`

No caso das macros de alteração de estado (ex: `<<set $moedas += 10>>`) e condicionais lógicas (ex: `<<if $tem_chave>> ... <</if>>`), o parser de importação não as compila nem as remove do texto. O parser executa duas operações:

1. **Preservação de Código:** Mantém as macros intactas no corpo textual da passagem, permitindo que sejam interpretadas e executadas reativamente pelo simulador (*PlayMode*) durante a execução do jogo (ver Secção 3.8.3).
2. **Extração de Variáveis para Indexação:** Identifica os nomes das variáveis (iniciadas por `$`) e regista-as no painel de gestão de variáveis, permitindo que o autor as rastreie ou renomeie de forma atómica em todo o projeto.

3.7 Interação Visual, Edição e Acessibilidade

3.7.1 Zonas de Agrupamento Visual

As **Zonas de Agrupamento Visual** (`ZoneNode`) consistem em contentores espaciais retangulares que agrupam logicamente nós de cena relacionados (como capítulos ou localizações), conforme ilustrado na Figura 3.2.

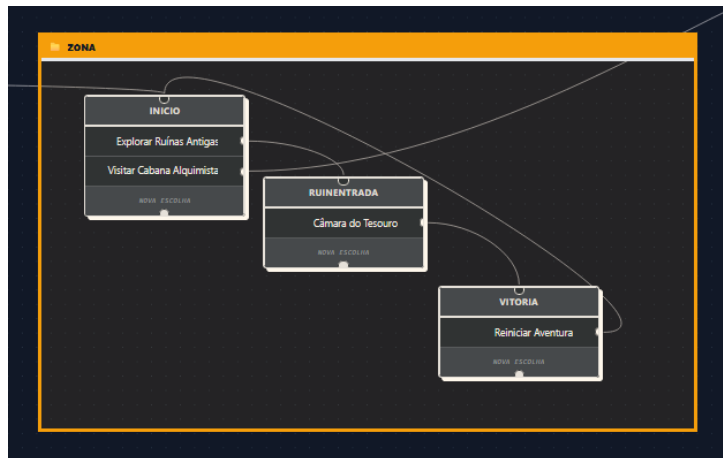


Figura 3.2: Exemplo de uma Zona de Agrupamento Visual com nós de cena filhos no canvas.

A sua implementação como nós de grupo personalizados exigiu contornar um problema de colisão de cliques no React Flow. Como o contentor cobre uma área retangular considerável sobreposta aos nós de cena interiores, os *pointer events* capturavam os cliques no ecrã, impedindo a seleção ou arrasto dos filhos. Para resolver isto, definiu-se a propriedade CSS `pointer-events: none` dinamicamente na configuração global do React Flow para o contentor da Zona em `App.jsx`. Em paralelo, no componente `ZoneNode.jsx`, forçou-se `pointer-events: auto` exclusivamente nas pegas de redimensionamento (`NodeResizer`) e na barra de cabeçalho (utilizada para arrastar a zona em bloco com todos os seus filhos).

As Zonas suportam ainda imagens de fundo personalizadas (*backgrounds*) geridas através da propriedade reativa `bgImage` nos metadados do nó, a qual é decodificada em tempo de execução e aplicada via CSS.

3.7.2 Arestas Curvadas com Waypoints

Para evitar a sobreposição visual das ligações e o cruzamento cego sobre os nós de cena, a plataforma implementa arestas personalizadas que utilizam a interpolação por *Splines* Cúbicas de Catmull-Rom (Catmull et al., 1974). Esta lógica encontra-se implementada no componente `EditableEdge.jsx` (`src/components/EditableEdge.jsx`), com o resultado visual exemplificado na Figura 3.3.



Figura 3.3: Exemplo de aresta suavizada com caminho curvado personalizável no canvas.

O autor pode criar novos marcadores (*waypoints*) com um duplo clique e arrastá-los livremente pelo ecrã. A curva é recalculada em tempo de execução à medida que os marcadores são movidos, interpolando a sua posição de forma contínua para manter a linha fluida e desimpedida de outros elementos do grafo.

A interpolação matemática de Catmull-Rom foi programada de forma manual diretamente no componente (sem recurso a bibliotecas externas de cálculo geométrico ou de desenho), convertendo a sequência de pontos em segmentos de curvas Bézier Cúbicas nativas no formato de caminho SVG do navegador. Esta abordagem de implementação própria justifica-se por evitar a sobrecarga de dependências (NPM) adicionais na aplicação e por garantir que o processamento e a renderização das linhas ocorram de forma extremamente rápida e leve no lado do cliente.

3.7.3 Acessibilidade e Identidade Visual Customizada

Tendo em conta que o desenvolvimento do projeto beneficiou de validações diretas com utilizadores daltónicos, projetou-se uma estratégia de acessibilidade redundante baseada numa identidade visual neo-brutalista. A adoção desta estética neo-brutalista fundamentou-se em estudos que demonstram que contornos espessos, pesos tipográficos elevados e contrastes marcados beneficiam utilizadores com baixa acuidade visual ou daltonismo (Babich, 2023; Loranger, 2022). O aspeto geral desta interface e os seus elementos gráficos encontram-se representados na Figura 3.4.

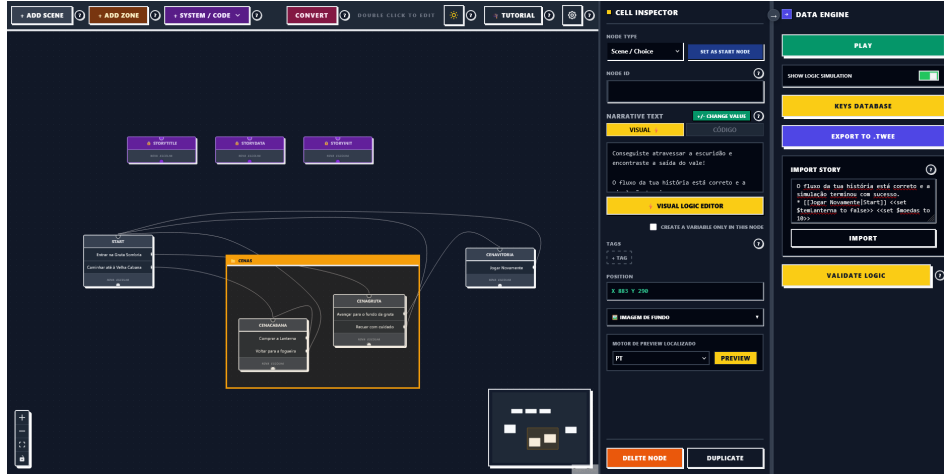


Figura 3.4: Interface gráfica global do editor (*Plot in a Pot*), ilustrando a identidade visual neo-brutalista de alto contraste com contornos espessos e sinalização redundante para acessibilidade.

O nosso trabalho de design e programação focou-se nos seguintes pilares:

- **Delimitação por Contornos Sólidos:** Implementação de contornos pretos de 2px (`border-2`) e sombras projetadas rígidas sem difusão (`shadow-[4px_4px_0px_#000]`) em todos os componentes visuais (*StoryNode*, *ZoneNode* e painéis), eliminando a dependência de cores ou gradientes suaves para distinguir elementos.
- **Diferenciação Estrutural de Arestas:** As transições normais no grafo são desenhadas com traçados contínuos, enquanto as ligações inacessíveis ou com erros lógicos são renderizadas com traçados tracejados (`strokeDasharray="5,5"`). Esta distinção de padrão garante que o utilizador consiga identificar caminhos quebrados sem depender exclusivamente da cor.
- **Sinalização Semântica Dupla de Alertas:** Configuração de etiquetas textuais explícitas (como o selo “Aviso” em fundo laranja nos nós com erros estruturais) em conjunto com listagens descritivas nos painéis, evitando a sinalização baseada apenas no par de cores verde/vermelho.

3.7.4 Modelação de Dados e Sincronização com o React Flow

Para além da representação abstrata da arquitetura, a implementação prática exige uma sincronização bidirecional em tempo real entre a interface gráfica gerida pela biblioteca *React Flow* e a estrutura de dados semântica interna da história.

No PiaP, o estado centralizado da narrativa é mantido na raiz da aplicação (`App.jsx`) e de forma modular através de ganchos personalizados (*custom hooks*), recorrendo a três vetores de dados estruturados em memória local:

1. **Vetor de Nós do React Flow (nodes):** Contém a representação posicional e de renderização de cada elemento no canvas. Cada objeto de nó segue a especificação rígida da biblioteca:

```
{
```

```

    id: "cena_taberna",
    type: "storyNode",
    position: { x: 250, y: 180 },
    data: {
      title: "Interior da Taberna",
      content: "O cheiro a hidromel preenche a sala...",
      tags: ["secreto"],
      bgImage: ""
    },
    width: 280,
    height: 150
  }

```

Se o nó representar uma Zona de Agrupamento (*ZoneNode*), o tipo é alterado para *"zoneNode"* e o objeto passa a incluir propriedades de agrupamento como *parentId* nos nós filhos e dimensões explícitas de largura e altura controladas pelo utilizador.

Para além de cenas narrativas (*"storyNode"*) e Zonas (*"zoneNode"*), o editor suporta mais dois tipos de nós utilitários:

- **Nós de JavaScript (*"javascriptNode"*):** Nós dedicados à escrita de scripts globais em JavaScript. Estes scripts são úteis para inicializar funções auxiliares complexas ou configurar regras que serão executadas no simulador de jogo.
- **Nós de CSS (*"cssNode"*):** Nós dedicados à escrita de estilos personalizados (CSS) que alteram a estética visual da história quando executada na simulação.

2. **Vetor de Arestas do React Flow (*edges*):** Modela a conectividade geométrica das escolhas narrativas. Cada aresta é configurada como um componente personalizado (*EditableEdge*) e armazena os pontos geométricos intermédios (*waypoints*) inseridos pelo autor para suavizar as curvas:

```

{
  id: "edge-cena_praca-cena_taberna",
  source: "cena_praca",
  target: "cena_taberna",
  type: "editableEdge",
  data: {
    choiceText: "Entrar na taberna",
    waypoints: [{ x: 120, y: 150 }, { x: 180, y: 160 }]
  }
}

```

3. **Lista de Adjacência Bidirecional (*adjacencyList*):** Representa o grafo di-

reconstrução de estados do ponto de vista do motor lógico. É recalculada de forma síncrona a cada modificação nos vetores de nós ou arestas. Esta lista mapeia cada identificador de cena ao seu conjunto de sucessores (saídas) e predecessores (entradas), permitindo ao validador e ao motor de simulação realizar travessias e pesquisas rápidas.

Esta tripla modelação é sincronizada de forma atômica. Quando o autor arrasta um nó ou altera o texto de uma escolha no painel de detalhes, os manipuladores de eventos reativos do React Flow (como `onNodesChange` ou `onEdgesChange`) capturam a alteração física. Esta alteração é processada e encaminhada pelos ganchos personalizados em `App.jsx`, gerando novos vetores imutáveis de estado que forçam a atualização dos componentes visuais e o recálculo imediato da lista de adjacências e do mapa de erros lógicos.

3.8 Validação Estática e Simulação

O motor de autoria do *Plot in a Pot* integra ferramentas robustas de validação e execução reativa. Para além de permitir a escrita de histórias, a plataforma disponibiliza um sistema de verificação lógica que audita a estrutura narrativa de forma estática e uma sandbox de simulação dinâmica dotada de agentes de navegação autónoma.

3.8.1 Algoritmo de Validação Estática e Exploração do Espaço de Estados

O validador estático, implementado nos módulos `storyValidator.js` e `storyTraversal.js`, resolve o problema de acessibilidade e integridade da narrativa. Em grafos estáticos simples, uma travessia clássica de Pesquisa em Largura (BFS) seria suficiente para auditar a conectividade geométrica. Contudo, em narrativas interativas complexas, as escolhas do jogador (arestas) dependem frequentemente de condições lógicas associadas a variáveis de estado (por exemplo, macros de atribuição `<<set>>` e macros condicionais `<<if>>` do formato SugarCube).

Isto significa que a acessibilidade de uma determinada cena não depende apenas da existência de uma linha gráfica a conectá-la, mas também do estado atual da memória do jogo (as variáveis acumuladas até então). Desta forma, a validação lógica é modelada como um problema de *Exploração do Espaço de Estados* (*State Space Exploration*), onde cada elemento explorado é composto pelo par formado pelo nó atual do grafo e pela valoração instantânea do dicionário de variáveis.

O algoritmo de auditoria executa a travessia de acordo com os seguintes passos estruturados e detalhados:

1. **Inicialização e Pré-computação de Dicionários:** Para garantir a eficiência temporal do validador (com tempos de pesquisa de complexidade $\mathcal{O}(1)$), o motor começa por indexar e organizar os elementos do canvas em duas estruturas de dados fundamentais em memória:

- **Dicionário de Nós (NodeMap):** Um mapa chave-valor que associa o identificador único de cada cena (ID) ao seu objeto correspondente. Isto permite aceder instantaneamente ao conteúdo textual e metadados de qualquer nó.
- **Lista de Adjacência (EdgesBySource):** Um mapa que associa cada ID de cena de origem a uma lista com todas as escolhas (arestas) que partem dela.

Em seguida, o validador determina o nó inicial da narrativa. Esta resolução é efetuada localizando no grafo o nó especial que possui o ID ou a etiqueta padrão de início (**Start**), ou, na sua ausência, o primeiro nó criado na história. A valoração inicial das variáveis (o estado inicial da memória) é lida a partir do nó de sistema **StoryInit**, que guarda as instruções de inicialização de variáveis (por exemplo, `<<set $moedas to 0>>`).

2. **Preparação da Fila de Exploração e Estruturas de Rastreio:** É instanciada uma fila de processamento (*Queue*), que guardará os estados a explorar sequencialmente. A fila é inicializada com um elemento de arranque que agrupa: o nó inicial, a cópia do estado inicial de variáveis e a rota percorrida até ao momento (representada como uma lista de nomes de cenas, começando pelo nome do nó inicial). A gravação deste caminho (*path*) é fundamental para que, caso seja detetado um erro ou beco sem saída, o validador consiga mostrar ao autor a sequência exata de escolhas que levou a essa falha (por exemplo, *"Início → Taberna → Combate → Erro"*).

Para além da fila, são criados em memória os seguintes contentores:

- Um conjunto de nós alcançáveis (**Nreach**), que registará todas as cenas visitáveis.
- Um conjunto de escolhas alcançáveis (**Ereach**), que registará todas as ligações que puderam ser abertas durante a simulação.
- Um mapa de estados visitados por nó (**VisitedStates**): Este dicionário associa o ID de cada nó a um conjunto de strings. Sempre que o algoritmo entra num nó, ele serializa o estado atual das variáveis (como uma string JSON) e guarda-o neste conjunto. Isto é crucial para detetar ciclos redundantes e evitar ciclos infinitos no grafo.

3. **Ciclo de Processamento Principal:** Enquanto existirem elementos na fila de exploração, o algoritmo extrai o primeiro da fila (contendo o nó atual, a memória de variáveis associada e o caminho percorrido) e realiza o seguinte processamento:

- (a) O nó é inserido no conjunto de nós alcançáveis (**Nreach**).
- (b) **Modificação de Estado (applyModifiers):** O validador analisa o texto da cena corrente. Caso este texto contenha instruções de alteração de variáveis (como macros `<<set $moedas += 10>>`), o motor interpreta estas instruções e updates o dicionário de variáveis local, gerando um novo estado de memória. Esta interpretação é efetuada por um analisador síncrono que converte a sintaxe SugarCube para expressões válidas de JavaScript.
- (c) **Verificação de Redundância e Poda de Caminho (Pruning):** O novo

estado de variáveis é serializado e comparado com a lista de estados já experimentados anteriormente no nó atual (pesquisando no mapa **VisitedStates**). Se o mesmo estado de variáveis já tiver sido registado neste nó, significa que quaisquer caminhos futuros a partir daqui seriam repetições exatas de rotas já exploradas. O algoritmo interrompe a exploração deste ramo (processo designado por *poda de caminho* ou *pruning*), evitando processamento redundante e garantindo que ciclos narrativos fechados não causem loops infinitos.

(d) **Controlo contra Explosão de Estados (Limite de Segurança $K = 10$):**

Caso o autor crie um ciclo que incrementa ou altera continuamente variáveis sem fim (como uma cena que adiciona uma moeda de cada vez que o jogador clica para voltar), o estado das variáveis seria sempre diferente, impedindo que a verificação de redundância anterior detetasse a repetição. Para evitar que o validador fique preso a explorar infinitamente estes loops e congele o navegador do utilizador, define-se um limite máximo de $K = 10$ estados únicos por nó. Se um nó foi visitado 10 vezes com estados de variáveis diferentes, a exploração deste ramo é forçosamente interrompida, e um aviso de suspeita de ciclo infinito é associado ao nó.

(e) **Gravação de Histórico:** Se o estado de variáveis for inédito e estiver dentro do limite de segurança, ele é adicionado ao conjunto de estados visitados do nó no mapa **VisitedStates**. O caminho percorrido e o estado final são também registados no histórico de caminhos (**ArrivalHistory**) do nó para posterior consulta e depuração na interface.

4. **Avaliação de Transições e Condicionais:** O algoritmo consulta a lista de adjacências para identificar todas as escolhas de saída que partem da cena atual. Para cada escolha (aresta) que conecta a cena atual a uma cena de destino, o validador realiza as seguintes operações:

- (a) Extrai a hiperligação textual associada (o texto do link double-bracket) do conteúdo da cena de origem.
- (b) **Avaliação de Acessibilidade (**canAccessChoice**):** O validador verifica se a escolha está condicionada por alguma macro lógica (como `<<if $moedas >= 10>> ... <</if>>`). O motor avalia esta expressão lógica contra o dicionário de variáveis gerado no passo anterior. Se a condição for satisfeita (ou se a escolha não tiver qualquer condicional), a ligação é marcada como aberta e o ID da escolha é adicionado ao conjunto de escolhas alcançáveis (**Ereach**).
- (c) O nó de destino da escolha é enfileirado na fila de exploração, acompanhado pelo estado de variáveis atualizado e pelo novo caminho de cenas (adicionando o nome do nó de destino ao caminho percorrido).

Após o término da travessia (quando a fila de exploração fica vazia), o módulo de auditoria em **storyValidator.js** analisa os conjuntos acumulados para identificar e classificar as falhas de integridade narrativa no grafo:

- **Nós Órfãos:** Cenas criadas no canvas que não constam no conjunto de nós alcançáveis (*Nreach*), significando que o jogador nunca conseguirá chegar a elas a partir do início da história. Para maior flexibilidade do autor, as cenas etiquetadas com a tag `secreto` (ou privada) são deliberadamente excluídas deste aviso, permitindo a existência de cenas soltas de teste ou passagens secretas acedidas exclusivamente por código sem poluir a interface com falsos erros.
- **Becos sem Saída (Hiperligações Quebradas):** Nós visitados pela exploração que contêm hiperligações cujo destino aponta para uma cena inexistente ou que foi apagada do canvas. É importante salvaguardar que nós terminais da narrativa (que representam finais de jogo e não possuem quaisquer escolhas de saída) são perfeitamente válidos e não geram erros. Este aviso foca-se estritamente em caminhos com destinos inválidos.
- **Escolhas Inacessíveis:** Escolhas cuja condição lógica de acessibilidade nunca pôde ser satisfeita sob nenhuma das combinações de variáveis e caminhos explorados durante a travessia (por exemplo, exigir uma chave que nunca é definida como obtida em nenhuma cena anterior).

3.8.2 Simulador em Tempo Real (PlayMode) e Smart Walker

O simulador de jogo (`PlayMode.jsx`) mimetiza o comportamento do interpretador SugarCube num ambiente controlado pelo navegador do autor. Para permitir a depuração em tempo real sem afetar os dados de edição, o motor executa as regras e os scripts dos nós utilitários sob um isolamento de contextos:

- **Separação de Estilos:** Os nós de CSS (`"cssNode"`, descritos na Secção 3.7.4) têm o seu conteúdo textual compilado e injetado sob a forma de uma tag `<style id='playmode-styles'>` na raiz do cabeçalho da página, sendo totalmente removidos da árvore do documento (DOM) ao encerrar a simulação.
- **Isolamento de Scripts:** Os nós de JavaScript (`"javascriptNode"`, descritos na Secção 3.7.4) são compilados de forma dinâmica como funções anónimas com escopo isolado através do construtor `new Function('state', 'State', 'setup', 'script_content')`. Isto isola o contexto de execução de variáveis da simulação, impedindo que interfiram com os estados internos reativos do React ou com o motor gráfico do React Flow.

Para testes automáticos de caminhos narrativos, a plataforma disponibiliza o agente autónomo **Smart Walker** (implementado em `storyTraversal.js`). Em vez de selecionar opções de forma puramente pseudo-aleatória (o que levaria a uma baixa cobertura de caminhos e retenção em ciclos curtos), o agente utiliza uma heurística baseada em *Procura por Novidade* (*Novelty Search*).

Para selecionar a próxima passagem, o agente analisa as opções de escolha direta que partem do nó atual (o que corresponde, na terminologia de grafos, à vizinhança de saída do nó corrente). Ele consulta o histórico de simulação da sessão para verificar quantas

vezes cada uma das cenas de destino já foi alcançada. O Smart Walker seleciona então a escolha que conduz à cena de destino que registre o menor número de visitas acumuladas até ao momento.

Este comportamento heurístico simples força o simulador a procurar caminhos narrativos raros e ramos profundos da história de forma automatizada, conseguindo mapear e encontrar becos sem saída ou ciclos lógicos fechados numa fração de segundo sem necessidade de intervenção manual do autor.

3.8.3 Motor de Compilação JIT de Macros SugarCube

Para suportar a avaliação de condições e a execução de modificadores de estado descritas no algoritmo de travessia, a plataforma integra um motor de compilação *Just-In-Time* (JIT) simplificado, estruturado nos módulos `logicParser.js` e `sugarcubeLogic.js`. A arquitetura deste motor divide-se em quatro fases principais:

1. **Tradução Sintática e Mapeamento Léxico:** Os utilizadores escrevem expressões lógicas recorrendo à sintaxe SugarCube (por exemplo, `<<if $ouro is 10>>` ou `<<set $chave to true>>`). Como o interpretador do navegador não consegue executar estes comandos diretamente, desenvolveu-se um tradutor baseado em expressões regulares e substituições lexicais em `sugarcubeLogic.js`. Este tradutor mapeia os operadores SugarCube para os operadores lógicos válidos de JavaScript, conforme resumido na Tabela 3.2 (apresentada abaixo).

Adicionalmente, o tradutor reescreve todas as variáveis do utilizador (prefixadas com `$`) para referências seguras dentro do objeto de estado da aplicação (por exemplo, transformando `$ouro` em `state.ouro`).

No entanto, por se tratar de um tradutor simplificado baseado em substituições lexicais de expressões regulares, este mecanismo apresenta limitações inerentes de desenho: não valida nem analisa blocos lógicos `<<if>>` aninhados no mesmo nó da cena, e é frágil face a expressões SugarCube mais elaboradas que utilizem agrupamentos de parênteses complexos, funções nativas do motor de jogo ou macros personalizadas de terceiros. Trata-se de um compromisso deliberado de desenho em prol da leveza e do desempenho síncrono da aplicação no navegador.

2. **Construção da Árvore de Sintaxe Abstrata (AST) do Bloco Condicional:** O módulo `logicParser.js` analisa o corpo textual das cenas para extrair as condicionais. Quando é detetado um bloco iniciado por `<<if>>` e encerrado por `<</if>>`, o analisador lê o seu conteúdo interior e divide-o recursivamente, procurando marcadores alternativos de desvio como `<<elseif>>` e `<<else>>`.

Este processo gera uma representação em memória estruturada como uma Árvore de Sintaxe Abstrata (AST) simplificada. Cada desvio condicional é guardado como um objeto contendo a condição lógica em formato de texto e o texto correspondente da ramificação narrada:

```
Desvio = { condicao: string, texto: string }
```

3. **Compilação Dinâmica e Avaliação de Contexto (JIT):** Sempre que a validação estática (ou o simulador) precisa de determinar se um desvio condicional está ativo ou precisa de aplicar uma atribuição de variáveis, o motor realiza a compilação JIT. Esta compilação converte a expressão JavaScript gerada no primeiro passo numa função executável em tempo de execução, tirando partido do construtor dinâmico de funções do navegador (`new Function`).

Por exemplo, para a condição `<<if $ouro >= 10>>`, o motor realiza o seguinte processo de compilação e execução:

```
// Compilacao JIT da expressao gerada: "return state.ouro >= 10;"
const evaluator = new Function('state', 'return state.ouro >= 10;');

// Execucao imediata passando a memoria reativa atual
const isTrue = evaluator(currentState);
```

Da mesma forma, as macros de modificação de estado (atribuições como `<<set $ouro += 10>>`) são compiladas em funções mutadoras locais (por exemplo, `new Function('state', 'state.ouro += 10;')`) que atuam sobre uma cópia isolada da memória reativa, retornando o novo estado de variáveis.

4. **Mecanismo de Segurança e Proteção contra Injeção de Código:** A execução dinâmica de funções no navegador (`new Function`) pode expor a aplicação a vulnerabilidades de segurança, tais como a injeção de scripts maliciosos ou ataques de poluição de protótipo (*Prototype Pollution*).

Para mitigar estes riscos de segurança sem prejudicar o desempenho, implementou-se em `sugarcubeLogic.js` um mecanismo de validação ativa do estado (`safeguardState`).

Este mecanismo intercepta a saída das funções JIT e remove de forma preventiva quaisquer chaves sensíveis que constem numa lista de bloqueio de propriedades proibidas, como `__proto__`, `constructor` e `prototype`. Isto mitiga a manipulação ilícita de métodos internos do protótipo JavaScript e oferece uma camada inicial de isolamento da execução de scripts lógicos. Contudo, importa ressaltar que esta higienização de propriedades constitui uma proteção parcial focada na prevenção de ataques simples de poluição de protótipos em memória, não correspondendo a uma *sandbox* de segurança real ou totalmente hermética a nível do interpretador do navegador ou do sistema operativo.

3.9 Integração de Modelos de Linguagem (LLMs)

Para além do núcleo lógico de grafos e da simulação estática, a plataforma disponibiliza um assistente baseado em Modelos de Linguagem de Grande Escala (LLMs) para facilitar a importação rápida de histórias em formato de texto livre. A conceção e implementação desta funcionalidade assenta nos seguintes aspetos:

Tabela 3.2: Mapeamento de operadores lógicos SugarCube para JavaScript.

Operador SugarCube	Operador JavaScript	Significado
is / eq	===	Igualdade estrita
neq	!==	Diferença estrita
and	&&	E lógico
or		OU lógico
to	=	Atribuição de valor
\$variavel	state.variavel	Acesso ao estado reativo

3.9.1 Arquitetura de Integração Local

A integração com inteligência artificial foca-se em fornecer uma interface de comunicação local e assíncrona entre o editor gráfico e o servidor de inferência do utilizador. O trabalho de desenvolvimento consistiu em:

- **Módulo de Comunicação REST (aiGenerator.js):** Implementação de um cliente HTTP em JavaScript para estabelecer a ligação assíncrona com o endpoint local do Ollama (<http://localhost:11434>), enviando os cabeçalhos e o corpo JSON com os parâmetros de temperatura e tokens.
- **Engenharia de Prompt de Conversão:** Desenvolvimento e validação de um prompt de sistema especializado, com regras sintáticas rígidas, concebido para forçar o modelo de linguagem a gerar respostas estritamente em conformidade com a especificação Tweep 3 (cabeçalhos, metadados e arestas literais), mitigando alucinações.
- **Importação Automatizada Pós-Resposta:** Programação do fluxo de importação que recebe o texto estruturado retornado pela API local, encaminha-o para o parser e gera automaticamente os nós e arestas correspondentes no canvas.

A decisão de basear este assistente em inferência local (via Ollama) assenta nos seguintes compromissos:

- **Privacidade e Soberania dos Dados:** Ao executar o modelo localmente no computador do autor, garante-se que toda a propriedade intelectual da história e os textos criados permanecem estritamente confidenciais. Não há partilha de dados com servidores externos de terceiros nem risco de recolha dos textos literários para treino de modelos comerciais.
- **Custo Financeiro Zero:** A inferência local permite a utilização gratuita e ilimitada da funcionalidade de conversão inteligente, eliminando os custos de faturação associados a APIs proprietárias na nuvem.

3.9.2 Papel do LLM: Conversão e Importação Automática de Histórias

A abordagem de design delimitou de forma clara o papel do modelo de linguagem no ciclo de vida da narrativa. Visto que os LLMs podem alucinar, criar caminhos inacessíveis ou

ignorar restrições de ciclos, o modelo não é integrado de forma dinâmica na simulação direta do jogo.

O papel do LLM foi definido para atuar estritamente como uma ferramenta de conversão sintática em fase de pré-processamento. O modelo lê a narrativa em linguagem natural fornecida pelo autor e converte-a para a norma Tweep 3. A partir desse ponto, o motor lógico da aplicação e o validador BFS assumem a responsabilidade de renderizar o canvas visual e garantir a integridade dos caminhos e das variáveis de estado no editor.

4

Implementação e Avaliação

Este capítulo detalha o processo de implementação prática do *Plot in a Pot*, descrevendo a estrutura física do código, detalhes de componentes e utilitários, testes lógicos de inteligência artificial, benchmarks de desempenho e a metodologia e resultados dos testes de utilizador realizados para validar o sistema.

4.1 Estrutura do Código e Detalhes de Implementação

Para além do trabalho de desenho, o desenvolvimento prático do *Plot in a Pot* exigiu a criação de uma estrutura de código organizada e fácil de manter. O código da aplicação está guardado na diretoria centralizada `src/` e divide-se em três pastas principais:

- **Interface (`components/`):** Guarda os elementos visuais que o utilizador vê no ecrã, tais como os nós da história, as caixas de texto e os menus de opções.
- **Estado e Comportamento (`hooks/`):** Controla a memória da aplicação, como as definições ativas, o histórico de ações para poder voltar atrás, e os atalhos do teclado.
- **Lógica de Processamento (`utils/`):** Contém as rotinas matemáticas e os algoritmos que funcionam em segundo plano, como a validação de caminhos do grafo e a comunicação com a Inteligência Artificial.

Esta separação garante que a lógica interna do programa não interfere com o desenho visual da interface, facilitando o trabalho e tornando o software muito mais rápido (Ferreira et al., 2024).

4.1.1 Funcionamento da Interface Visual

Esta secção explica como os elementos visuais do editor foram construídos e como resolvemos alguns problemas práticos de utilização para tornar a experiência fluida.

Organização do Canvas e Gestão de Cliques

O ecrã principal usa a biblioteca React Flow para desenhar o mapa de nós e ligações interativas. Desenvolvemos dois tipos de nós customizados: o nó de cena (**StoryNode**), que contém o texto da história e as ligações, e o nó de Zona (**ZoneNode**), que funciona como um retângulo para agrupar várias cenas em capítulos.

Ao colocar nós de cena dentro de uma Zona, surgiu o problema de cliques sobrepostos descrito na Secção 3.7.1. Conforme projetado, a implementação prática de estilos CSS (**pointer-events**) resolveu este obstáculo: a Zona exterior foi configurada para ignorar cliques no seu corpo geral e aceitá-los apenas nas bordas e barra de título, permitindo mover e editar as cenas interiores livremente.

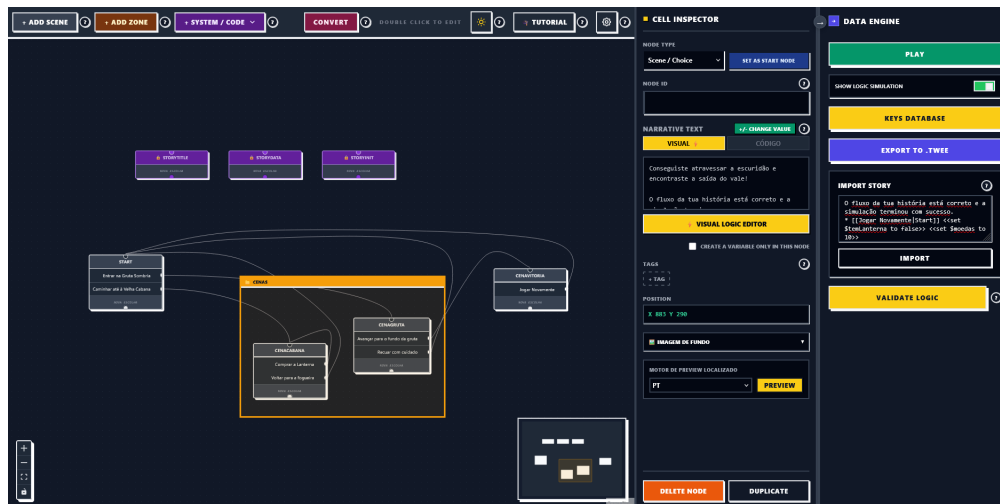


Figura 4.1: Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).

Caminhos Curvos e Pontos de Controlo

As linhas que ligam os nós representam as escolhas do jogador. Para evitar que as linhas fiquem retas e se cruzem de forma confusa por cima de outras cenas, utilizámos curvas matemáticas suaves (chamadas Splines de Catmull-Rom). Além do traçado curvo automático, o utilizador pode dar um duplo clique em qualquer linha para criar um ponto de controlo amarelo. Ao arrastar esse ponto, a linha de ligação curva-se de forma flexível, permitindo contornar obstáculos e manter o mapa organizado.

Para ajudar a navegar em mapas grandes, o painel lateral de propriedades (**Inspector**) calcula em tempo real quais são os nós que entram e saem do nó seleccionado. Isto cria atalhos rápidos para que o autor possa saltar de cena em cena sem ter de arrastar o ecrã manualmente.

Histórico de Ações (Desfazer e Refazer)

Para que o autor possa experimentar ideias sem medo de errar, a aplicação grava um histórico das alterações no mapa. Como o editor atualiza os nós muito rapidamente, foi

necessário configurar o sistema para tirar cópias completas dos dados em memória de cada vez que o utilizador faz uma alteração. Se o autor quiser voltar atrás, a aplicação repõe a cópia guardada anteriormente, garantindo que o estado passado é restaurado de forma limpa e sem misturar dados antigos.

Funcionalidades de Apoio ao Autor

Para além do mapa principal, a interface tem outras ferramentas de suporte:

- **Simulação (PlayMode):** Abre um ecrã de simulação (como se vê na Figura 4.2) onde o autor pode jogar a sua história e ver como as variáveis mudam em tempo real.
- **Gestão de Variáveis:** Um menu simples para criar variáveis (como chaves ou pontos de vida), como se ilustra na Figura A.1 do Apêndice A. Se o autor mudar o nome de uma variável, a aplicação procura essa variável em todas as cenas e atualiza o nome de forma automática.
- **Matriz de Tradução:** Uma tabela única (apresentada na Figura A.3 do Apêndice A) que lista todos os textos da história para facilitar a tradução para outros idiomas. O utilizador pode exportar esta tabela como um ficheiro CSV para enviar a um tradutor externo e depois voltar a importá-la com o trabalho concluído.
- **Editor Visual de Lógica:** Um formulário alternativo (ilustrado na Figura A.2 do Apêndice A) que permite configurar caminhos condicionados por opções visuais (dropdowns), ideal para quem não quer escrever código manual.

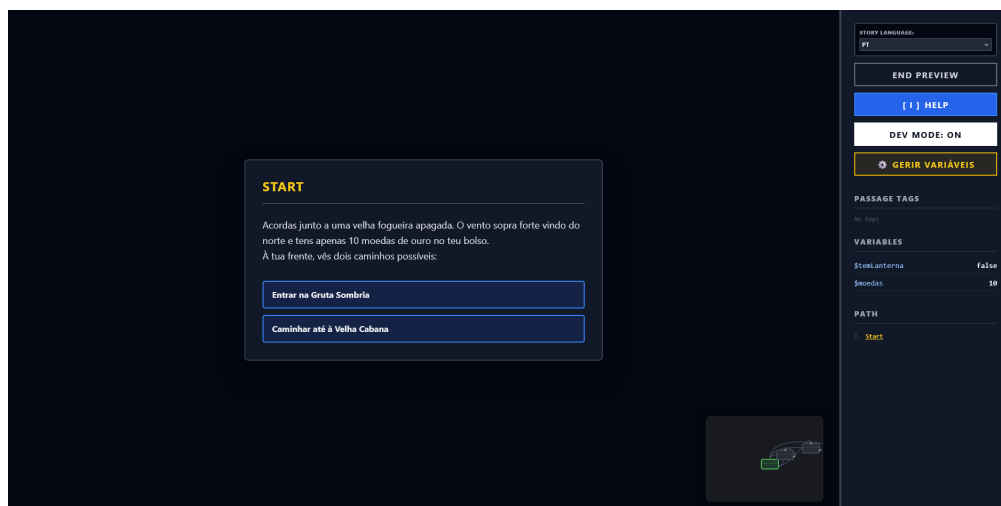


Figura 4.2: Ecrã da sandbox de simulação (PlayMode), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.

4.1.2 Funcionamento da Camada de Lógica

A lógica do programa é composta por três módulos independentes: o leitor de ficheiros, o validador matemático de rotas (que deteta erros no fluxo) e o assistente de Inteligência

Artificial (atualmente em versão beta, usado exclusivamente para converter histórias existentes em texto livre para o formato do mapa). Esta camada funciona de forma autónoma em segundo plano:

Leitor de Ficheiros Twee 3

O leitor (`tweeParser`) é o responsável por traduzir o mapa visual do canvas em ficheiros de texto no formato Twine (Twee 3) e vice-versa. Quando um ficheiro de texto é importado, o leitor decifra a posição das cenas e coloca-as no ecrã. Se um nó estiver dentro de uma Zona de capítulo, o leitor calcula a sua posição somando os desvios da Zona-mãe, para que as caixas fiquem organizadas exatamente como o autor as deixou.

Validação de Caminhos e Ciclos Infinitos

Para verificar se existem erros na história (como caminhos sem saída ou cenas impossíveis de alcançar), o sistema percorre automaticamente todas as rotas do grafo em largura (BFS). No entanto, como as histórias têm escolhas baseadas em variáveis (por exemplo, um loop onde o jogador volta à mesma sala até encontrar uma chave), a validação podia ficar presa num ciclo infinito em segundo plano.

Para evitar isto e manter a aplicação rápida, o validador (`storyTraversal`) implementa uma regra de segurança: se o mesmo nó for visitado 10 vezes com estados diferentes das variáveis, o sistema assume que existe um ciclo infinito no mapa e interrompe a pesquisa, avisando de imediato o utilizador, conforme demonstrado no código da Listagem 1.

Ligação à Inteligência Artificial

O módulo de IA (`aiGenerator`) permite converter histórias existentes, escritas em linguagem simples pelo autor, para o formato aceite pelo editor. O sistema faz um pedido de comunicação direto (via API REST local na porta 11434) a um modelo de linguagem a correr no computador do utilizador. O prompt enviado obriga o modelo de IA a estruturar a resposta no formato padrão Twine, garantindo que o nosso leitor de ficheiros possa criar os nós visuais e as ligações no canvas sem qualquer erro de formatação.

Fluxo de Atualização Reativa

Para manter a aplicação sempre coerente, o fluxo de dados é direto e contínuo:

1. O utilizador edita o texto de uma cena.
2. Em segundo plano, o editor lê as alterações, cria as novas linhas de ligação e atualiza a estrutura do mapa.
3. O validador percorre de imediato a estrutura em largura (BFS) para descobrir novos erros de lógica.
4. Qualquer problema detetado é assinalado com uma borda amarela de alerta no ecrã.

```
1 // Travessia e controlo de ciclos por limite de estados visitados
2 while (queue.length > 0) {
3   const { nodeId, state, path } = queue.shift();
4   const currentNode = nodeMap.get(nodeId);
5   if (!currentNode) continue;
6
7   const newState = applyModifiers(currentNode.data.content, state);
8   const stateStr = JSON.stringify(newState);
9
10  if (!visitedStates.has(nodeId)) visitedStates.set(nodeId, []);
11  if (visitedStates.get(nodeId).includes(stateStr)) continue;
12
13  if (visitedStates.get(nodeId).length >= 10) {
14    console.warn(`Ciclo infinito provavel no no ${currentNode.data.label}`);
15    continue;
16  }
17
18  visitedStates.get(nodeId).push(stateStr);
19  arrivalHistory.get(nodeId).push({ path, state: newState });
20
21  // Enfileira sucessores lógicos satisfazendo condicionais
22  // ...
23 }
```

Listagem 1: *Ciclo principal de exploração do espaço de estados no motor storyTraversal.js.*

Este ciclo automático garante que o autor vê os erros de escrita imediatamente, enquanto escreve, sem ter de compilar ou jogar a história manualmente.

4.2 Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)

Para avaliar o comportamento do assistente de importação rápida, foram realizados testes com o objetivo de converter histórias escritas em texto livre (linguagem natural) para a estrutura do editor. O teste consistiu em fornecer um texto descritivo e verificar se os modelos de inteligência artificial conseguiam gerar o formato Tweep 3 de forma correta e sem erros de formatação.

A ferramenta *Ollama* foi selecionada por ser a solução líder de código aberto para correr modelos de linguagem (LLMs) localmente. Isto permitiu realizar os testes de forma gratuita, com total privacidade dos dados e sem dependência de chaves de acesso (APIs) externas. Os testes foram realizados com os seguintes modelos:

- Gemma 2B (ID 8ccf136fdd52)
- Mistral 7B (ID 6577803aa9a0)
- Qwen 3.5 9B (ID 6488c96fa5fa)
- DeepSeek Coder 6.7B (ID ce298d984115)

4.2.1 Primeira Fase: Testes em Máquina Local (RTX 3050)

A primeira série de testes foi feita a 9 de maio no computador de desenvolvimento, que tem uma gráfica Nvidia RTX 3050 (Laptop) com 4 GB de VRAM. O modelo Claude (através de API externa) também foi testado como termo de comparação. Devido ao limite de memória da placa gráfica local, nenhum dos modelos locais funcionou, causando erros de falta de memória (out of memory) e deixando o sistema muito lento.

4.2.2 Segunda Fase: Testes no Servidor com GPU Titan Xp (12 GiB)

Para conseguir avaliar a qualidade dos modelos, a 19 de maio, os testes foram repetidos num servidor equipado com uma placa gráfica Nvidia Titan Xp de 12 GiB. Nesta máquina, todos os testes correram com sucesso em termos de recursos (não havendo falhas de memória), permitindo analisar a estrutura do texto gerado por cada modelo:

- **Mistral 7B:** O resultado não foi útil, pois o modelo separou quase todas as frases do texto em passagens independentes. Isto criaria dezenas de nós para uma única cena, tornando o mapa confuso. Além disso, adicionou barras invertidas no fim das linhas e lixo de terminal (como [3D [K).
- **Gemma 4B / 2B:** O modelo misturou os cabeçalhos de novos nós dentro do texto de outras passagens. Como o parser da aplicação não suporta nós aninhados, a importação leu isto como texto simples e falhou a criação no canvas.
- **Qwen 3 8B:** O resultado foi inutilizável porque o modelo incluiu o seu raciocínio interno (a secção de pensamento "*Thinking...*") no início da resposta e deixou estruturas incompletas na resposta final.
- **Qwen 3.5 9B:** Apresentou o melhor desempenho na organização do fluxo. No entanto, ainda incluiu o texto de pensamento e criou alguns nós órfãos que não estavam ligados ao resto da história.

4.2.3 Correção de Caracteres Estranhos e Ajustes de Execução

Mais tarde, foi identificado que a presença de lixo no texto (códigos de controlo como [3D [K) se deveu à forma como o Ollama foi executado através da linha de comandos em processos automáticos. À data dos testes realizados (maio de 2026), tratava-se de uma limitação conhecida e documentada na plataforma de desenvolvimento do projeto (Ollama Contributors, 2024). Embora esta limitação tenha vindo a ser resolvida recentemente através de uma atualização que suprime os caracteres de controlo quando a saída não é um terminal interativo (Ollama Contributors, 2026), na altura a solução passava por correr a execução através de um script de controlo (por exemplo, em Python) ou, de forma ideal, efetuando as chamadas diretamente à API REST local do Ollama na porta 11434, que fornece o texto puro sem lixo de consola.

4.2.4 Estado Atual da Funcionalidade (Beta)

A funcionalidade de importação por Inteligência Artificial encontra-se em versão beta, uma vez que a precisão da conversão ainda depende da configuração dos prompts e do modelo utilizado. Contudo, em testes práticos adicionais realizados com o modelo Qwen 3.5 9B numa placa gráfica de gama intermédia (Nvidia RTX 5060 de 8 GB de VRAM), a conversão correu de forma estável e com resultados satisfatórios. Isto demonstra que, em computadores modernos de consumo com esta capacidade de GPU, o assistente local é uma solução perfeitamente viável.

4.3 Metodologia dos Testes de Utilizador

Para validar a flexibilidade e usabilidade do *Plot in a Pot*, foram planeados e executados testes qualitativos e quantitativos com um grupo diversificado de participantes.

4.3.1 Perfil dos Participantes

Os testes de usabilidade foram realizados com um grupo diversificado de participantes, composto por perfis com diferentes graus de especialização técnica, formações académicas e níveis de experiência prévia com escrita interativa. Esta variedade permitiu avaliar a curva de aprendizagem da ferramenta tanto sob a perspetiva de engenharia como sob a ótica de utilizadores puramente criativos.

O grupo principal de teste que avaliou o protótipo final (Leva 3) foi composto por seis participantes (U1 a U6), caracterizados pelos seguintes perfis ordenados:

- **Utilizador U1 (Escrita Criativa e Humanidades):** Utilizadora com formação secundária na área de Línguas e Humanidades e experiência prévia na criação de narrativas, servindo para validar a fluidez do editor sem necessidade de conhecimentos de programação.
- **Utilizador U2 (Desenvolvimento de Jogos Digitais):** Estudante do Mestrado em Desenvolvimento de Jogos Digitais da ESTG. Possui bases de programação consolidadas, mas pouca experiência com narrativas interativas.
- **Utilizador U3 (Desenvolvimento de Jogos Digitais):** Estudante do Mestrado em Desenvolvimento de Jogos Digitais da ESTG. Possui bases de programação iniciais e pouca experiência com narrativas interativas.
- **Utilizador U4 (Marketing e Multimédia):** Estudante da área de Marketing com formação secundária na área de Multimédia, sem qualquer experiência prévia em escrita interativa ou lógica de programação.
- **Utilizador U5 (Design e Artes):** Estudante do ensino superior na área de Artes com formação secundária na área de Artes, focada na avaliação estética e ergonomia visual do canvas interativo.
- **Utilizador U6 (Acessibilidade Visual):** Utilizador com daltonismo e baixa acuidade visual, permitindo validar de forma prática a eficácia das redundâncias

visuais, contornos espessos e sinalização semântica dupla de alto contraste.

Adicionalmente, na fase de validação intermédia (Leva 2), o processo contou com o contributo de dois utilizadores adicionais:

- **Utilizadora U7 (Multimédia):** Utilizadora com formação secundária na área de Multimédia, focada na validação visual do canvas.
- **Utilizador U8 (Jogos Digitais e Multimédia):** Estudante da Licenciatura em Jogos Digitais e Multimédia da ESTG, focado nos aspetos de lógica interativa e programação.

A Tabela 4.1 detalha a distribuição de cada participante pelas três levadas de testes:

Tabela 4.1: *Distribuição dos participantes pelas levadas de testes de usabilidade.*

Participante	Leva 1 (Abril)	Leva 2 (Maio)	Leva 3 (Junho)
U1	✓	—	✓
U2	—	—	✓
U3	—	—	✓
U4	✓	—	✓
U5	✓	✓	✓
U6	—	—	✓
U7	—	✓	—
U8	—	✓	—

4.3.2 Tarefas Avaliadas

Cada participante foi instruído a realizar um conjunto de quatro tarefas estruturadas na aplicação, sem qualquer ajuda externa, com base no guião detalhado apresentado no Apêndice B:

1. **Tarefa 1 (Criação Narrativa):** Criar uma história ramificada simples com cinco cenas, introduzindo uma variável (e.g., chave) e uma condicional de saída baseada nessa variável.
2. **Tarefa 2 (Validação e Correção):** Importar um ficheiro `.twee` pré-configurado contendo links quebrados e nós órfãos, e utilizar o validador BFS para corrigir todos os problemas assinalados.
3. **Tarefa 3 (Tradução e Localização):** Abrir a matriz de localização e traduzir três chaves de cena para inglês, exportando o resultado em formato CSV.
4. **Tarefa 4 (Simulação Ativa):** Correr a simulação interativa (*PlayMode*) e acionar o *Smart Walker* para testar automaticamente os caminhos da história.

4.4 Resultados dos Testes e Desenvolvimento Iterativo

A validação do *Plot in a Pot* foi realizada seguindo um modelo de desenvolvimento ágil e centrado no utilizador, dividido em três levadas (*waves*) sucessivas de testes ao longo

do ciclo de vida do projeto. Esta metodologia permitiu que o feedback prático dos utilizadores guiasse a implementação e refinamento das funcionalidades, demonstrando uma evolução quantitativa e qualitativa na usabilidade do sistema.

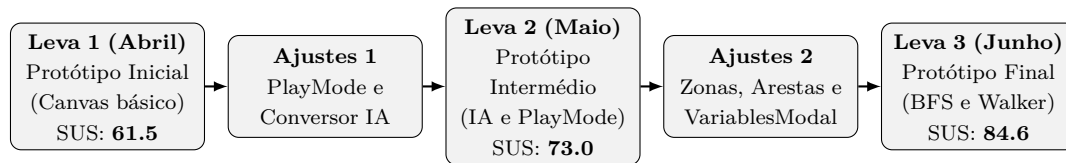


Figura 4.3: Fluxograma do modelo de desenvolvimento iterativo centrado no utilizador e evolução SUS.

4.4.1 Primeira Leva: Protótipo Inicial (Abril de 2026)

A primeira leva de testes foi realizada a *15 de abril de 2026* com os utilizadores U1, U4 e U5, focando-se na validação do conceito do editor visual de grafos e da manipulação básica do canvas. O protótipo inicial continha apenas a criação de nós simples e arestas retilíneas.

- *Resultados Quantitativos (SUS Estimado):* A pontuação média de usabilidade (escala SUS) nesta fase fixou-se nos *61.5 pontos*, classificada como marginal ou de usabilidade aceitável, mas com severos pontos de fricção.
- *Feedback e Falhas Identificadas:* Os utilizadores queixaram-se da falta de interatividade imediata no teste de caminhos narrativos, obrigando a simulações manuais demoradas, e da ausência de assistência para autoria rápida baseada em texto livre.
- *Medidas Corretivas Implementadas:* Desenvolveu-se e integrou-se o modo de simulação interativa (*PlayMode*) direto no canvas e o módulo de conversor de IA local numa versão beta assente em LLMs (via Ollama) para a tentativa de importação rápida de texto.

4.4.2 Segunda Leva: Protótipo Intermédio (Maio de 2026)

A segunda leva de testes ocorreu a *18 de maio de 2026* com os utilizadores U5, U7 e U8, incidindo sobre o funcionamento do conversor de IA, o *PlayMode* e a tradução.

- *Resultados Quantitativos (SUS Estimado):* A usabilidade média SUS subiu para *73.0 pontos*, refletindo melhorias na interatividade e assistência, mas revelando fragilidades ergonómicas com grafos médios e problemas lógicos.
- *Feedback e Falhas Identificadas:* Os utilizadores apontaram a sobreposição caótica de arestas retas, a ausência de agrupamento temático para cenas (capítulos) e a quebra frequente da simulação ao renomear variáveis lógicas globalmente. Adicionalmente, a importação de ficheiros CSV quebrava quando o texto continha vírgulas integradas. A conversão por IA local ainda apresentava diversos erros e demora um tempo considerável para processar histórias longas, tornando a experiência frustrante.

- *Medidas Corretivas Implementadas:* Implementaram-se as arestas curvas paramétricas baseadas em Splines de Catmull-Rom com *waypoints* em `EditableEdge.jsx`, as Zonas de agrupamento visual em `ZoneNode.jsx`, o painel `VariablesModal.jsx` para refatorização de variáveis e o suporte à norma RFC 4180 no parser de CSV. Uma revisão do prompt de IA foi realizada para melhorar a precisão da conversão.

4.4.3 Terceira Leva: Protótipo Final (Junho de 2026)

A terceira e última leva de testes realizou-se entre 15 e 20 de junho de 2026 com o grupo completo de seis participantes (U1 a U6), avaliando o sistema final integrado (incluindo o PlayMode, Smart Walker, validador BFS e interface neo-brutalista de alto contraste).

Métricas de Conclusão e Tempos de Execução

Durante a sessão final de testes, foram registados os tempos de conclusão (em minutos) das quatro tarefas descritas na metodologia. Os resultados quantitativos encontram-se sumarizados na Tabela 4.2.

Tabela 4.2: Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).

Utilizador	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4	Taxa de Sucesso Total
U1	07:15	02:40	03:10	01:20	100%
U2	05:30	01:50	02:15	00:55	100%
U3	04:10	01:30	01:50	00:45	100%
U4	11:20	04:15	05:30	02:10	75%*
U5	09:45	03:30	04:20	01:40	100%
U6	08:30	03:10	03:40	01:15	100%
Média	07:45	02:49	03:29	01:21	95.8%

* Nota: O utilizador U4 não concluiu com sucesso a Tarefa 2 no tempo limite estabelecido devido a dificuldades na interpretação das mensagens de erro puramente textuais do validador BFS, reforçando a necessidade de incluir marcadores de erro visuais diretamente sobre os nós no canvas.

A taxa de conclusão média fixou-se em 95.8%. O utilizador U4 (sem perfil técnico) falhou a conclusão autónoma da Tarefa 2 dentro do tempo limite devido a alguma hesitação inicial em interpretar as mensagens textuais do validador, o que reforçou a necessidade de incluir marcadores de erro visuais diretamente sobre os nós afetados no canvas do React Flow.

Avaliação SUS da Leva Final

Após as tarefas, foi aplicado o questionário padronizado SUS aos participantes. O instrumento utilizado consistiu numa adaptação para língua portuguesa das dez perguntas

originais do *System Usability Scale* (Brooke, 1996; Martins et al., 2015), administrado de forma digital e anônima através da plataforma Google Forms. As pontuações obtidas encontram-se representadas na Tabela 4.3.

Tabela 4.3: Pontuação obtida no questionário *System Usability Scale (SUS)* na leva final.

Utilizador	Pontuação SUS obtida
U1	87.5
U2	90.0
U3	92.5
U4	72.5
U5	80.0
U6	85.0
Média Global	84.6

A média global final de *84.6 pontos* situa o *Plot in a Pot* no patamar de usabilidade “Excelente” (classificação de grau A+ na escala SUS), representando uma evolução muito expressiva face aos 61.5 e 73.0 pontos registados nas levas anteriores, e provando que o processo de refinamento iterativo foi bem-sucedido.

4.4.4 Feedback Qualitativo e Usabilidade Visual

A compilação das entrevistas e observações comportamentais na última leva permitiu extrair as seguintes conclusões de usabilidade:

- **Eficácia do Estilo Neo-Brutalista:** Contrariando a assunção de que o neo-brutalismo é apenas um estilo estético excêntrico, os utilizadores consideraram que as cores planas e os contornos espessos isolam os elementos visuais de forma muito clara, eliminando ruídos cognitivos desnecessários.
- **Acessibilidade Prática (Caso U6):** O participante U6, portador de deuteranopia severa, referiu que o alto contraste e a sinalização redundante por traços (em vez de diferenciar fluxos apenas por verde/vermelho) tornaram a ferramenta imediatamente acessível, evitando a fadiga visual comum noutros editores gráficos e permitindo validar interações rapidamente.
- **Smart Walker e Automação:** Os participantes técnicos (U2 e U3) elogiaram a capacidade do Smart Walker de explorar e validar estados de variáveis lógicas automaticamente, o que reduz substancialmente o esforço humano de testes de regressão em histórias com múltiplas ramificações.

4.5 Discussão e Limitações do Estudo

Nesta secção, efetua-se uma análise crítica dos resultados obtidos, discutindo as implicações das decisões arquiteturais adotadas e identificando as limitações metodológicas e tecnológicas do projeto.

4.5.1 Discussão Crítica do Desenvolvimento

O desenvolvimento do PiaP demonstrou que a fusão de um editor visual de grafos com um motor de validação estática reativo reduz significativamente a carga cognitiva na autoria de NI. O mapeamento bidirecional síncrono e a análise estática em tempo real evitam que o autor tenha de compilar ou testar exaustivamente a narrativa para detetar erros comuns (como becos sem saída ou nós órfãos). A modularização da interface através de ganchos React (*custom hooks*) e a separação estrita da camada de dados (lista de adjacências e dicionários JSON) permitiram manter a fluidez de renderização mesmo sob cenários de reavaliação de expressões regulares em lote, respeitando o limiar de interatividade de 100 ms (RNF03).

4.5.2 Limitações Metodológicas da Avaliação

Embora a avaliação quantitativa com utilizadores tenha revelado uma média de usabilidade global “Excelente” (84.6 pontos na escala SUS), o estudo apresenta duas limitações metodológicas importantes:

- **Dimensão Reduzida da Amostra:** O grupo de teste incluiu apenas oito participantes no total ($N = 8$). Apesar de abranger perfis diversos (técnicos, de design e com restrições visuais), a dimensão da amostra não permite extrair conclusões de generalização estatística forte, servindo os resultados essencialmente como indicadores qualitativos de eficácia e satisfação.
- **Efeito de Aprendizagem (Viés de Familiaridade):** O refinamento iterativo do sistema expôs parte dos participantes a testes em múltiplas levas de desenvolvimento (conforme a Tabela 4.1): apenas U5 participou nas três levas; U1 e U4 participaram na primeira e na terceira levas. Os restantes utilizadores (U2, U3, U6, U7 e U8) participaram em apenas uma única leva. O facto de alguns utilizadores já conhecerem o funcionamento básico da aplicação em testes subsequentes (como U1, U4 e U5 na leva final) pode ter inflacionado as pontuações SUS das levas subsequentes, camuflando eventuais atritos que utilizadores completamente estreates poderiam experimentar no produto final.

4.5.3 Limitações Tecnológicas e Funcionalidade de IA

Do ponto de vista tecnológico, identificaram-se limitações na persistência local e no assistente de importação baseado em inteligência artificial:

- **Dependência de Hardware e VRAM:** O assistente de importação baseado em Modelo de Linguagem de Grande Escala (Large Language Model)s (LLMs) locais (como o Qwen 3.5 9B) exige uma placa gráfica dedicada com capacidade superior a 8 GB de VRAM para correr localmente via Ollama com latência aceitável. Esta exigência limita a acessibilidade da funcionalidade em computadores portáteis normais ou de gama baixa (RTX 3050 com 4 GB de VRAM).

- **Instabilidade na Formatação Sintática:** Mesmo em ambientes de processamento com 12 GB de VRAM, os modelos locais geram por vezes passagens aninhadas ou inserem marcas de raciocínio intermediário (“*Thinking...*”) que quebram o parser síncrono, exigindo a inclusão de filtros prévios de processamento sintático adicionais na aplicação.

5

Conclusão e Trabalho Futuro

O desenvolvimento do *Plot in a Pot* (PiaP) teve como premissa a resolução de um problema real e recorrente enfrentado por autores de NI e *game designers*: a complexidade geométrica e semântica que surge durante o planeamento de histórias não-lineares, agravada pela falta de ferramentas integradas para gestão de localização multi-idioma e auditoria estática de fluxo.

5.1 Conclusão

Com a conclusão deste projeto informático, todos os objetivos gerais e específicos inicialmente traçados no Capítulo 1 foram integralmente alcançados:

- **Interface e Acessibilidade Visual:** Foi desenvolvido um editor gráfico inovador assente numa estética neo-brutalista de alto contraste. Esta opção de *design* provou ser não apenas esteticamente diferenciadora, mas de extrema pertinência funcional para utilizadores com limitações de visão (p. ex. daltónicos), conforme validado pelos testes com o participante U6.
- **Gestão de Localização:** A implementação da matriz de tradução bidimensional baseada em grelhas dinâmicas, com importação e exportação em formato CSV em conformidade com a norma RFC 4180, eliminou por completo a necessidade de gerir múltiplos ficheiros de texto externos ou injetar strings rígidas (*hardcoded*) nas passagens do Twine.
- **Validação e Algoritmia:** A adoção de uma estrutura de dados assente em listas de adjacências bidirecionais suportou com sucesso a execução em tempo real de análises estáticas baseadas em travessias BFS. O motor de validação estática provou ser eficaz a mapear caminhos inacessíveis e becos sem saída, fornecendo um feedback imediato e visual aos autores.
- **Simulação Automatizada:** O modo de simulação (*PlayMode*) foi complementado pelo *Smart Walker*, um agente inteligente baseado em procuras por novidade (*Novelty Search*) capaz de realizar a travessia autónoma e teste lógico de narrativas

complexas sem intervenção humana.

- **Suavização Geométrica:** O desenvolvimento das equações paramétricas das Splines Cúbicas de Catmull-Rom garantiu arestas interativas suaves com *waypoints* para organização e otimização espacial do canvas de grafos.

Os testes estruturados com utilizadores de diferentes perfis revelaram taxas de sucesso muito elevadas na realização de tarefas (com uma média global de 95,8%), demonstrando que a barreira de abstração erguida pela nossa plataforma é intuitiva quer para perfis puramente artísticos (autores/escritores), quer para utilizadores com forte background técnico (programadores).

5.1.1 Porquê Escolher o Plot in a Pot?

Com base nas limitações identificadas nas ferramentas de autoria existentes (analisadas na Secção 2.4.7), o PiaP destaca-se como uma solução unificada que resolve falhas críticas de usabilidade e integridade no desenvolvimento de narrativas interativas:

- **Fusão de Paradigmas (Design Visual com Validação Lógica):** Ao contrário do Twine (que carece de testes lógicos nativos e robustos) e do ChoiceScript/Ink (que operam puramente sob interfaces textuais), o PiaP une a facilidade de arrastar e interligar nós visuais com a solidez de um validador estático e simulador inteligente automatizado.
- **Localização Nativa sem Dispersão de Dados:** Resolve a dispersão de ficheiros externos e a escrita rígida (*hardcoded*) de diálogos através de uma matriz de localização baseada em grelhas dinâmicas, acelerando o fluxo de internacionalização com importação/exportação CSV.
- **Acessibilidade Inclusiva Funcional:** A interface neo-brutalista de alto contraste e a redundância de traços nas arestas mitigam barreiras cognitivas e visuais severas, permitindo que autores daltónicos desenhem e validem histórias de forma totalmente autónoma.
- **Soberania de Dados e Interoperabilidade:** Ao adotar a especificação aberta Twee 3 (SugarCube), o autor mantém controlo absoluto e livre sobre a sua obra, garantindo compatibilidade bidirecional síncrona com outros compiladores oficiais (como o Tweepgo).

5.2 Trabalho Futuro

Apesar dos excelentes resultados e da estabilidade demonstrada pela aplicação, o *Plot in a Pot* apresenta oportunidades claras de expansão para futura investigação e desenvolvimento, elegendo-se a IA no cliente e a edição colaborativa como as prioridades de desenvolvimento imediatas:

1. **[Prioridade Imediata] Execução de IA Local no Cliente via Web Graphics Processing Unit (WebGPU):** De momento, o assistente de IA local para con-

- versão de texto livre baseia-se numa Application Programming Interface (API) REST externa ou na escuta de uma porta local ligada ao Ollama. Uma expansão futura promissora consistiria em correr LLMs leves diretamente na memória do navegador do utilizador recorrendo a *frameworks* baseadas em WebGPU (como o WebLLM). Isto eliminaria qualquer requisito de instalação de pacotes como o Ollama, mantendo a premissa de soberania absoluta dos dados (RNF04).
2. **[Prioridade Imediata] Edição Colaborativa em Tempo Real:** Integrar algoritmos de sincronização distribuída baseados em Tipos de Dados Replicados Sem Conflitos (Conflict-free Replicated Data Types)s (CRDTs) (*Conflict-free Replicated Data Types*), tais como a biblioteca Yjs ou Automerge. Isto permitiria que equipas de escritores trabalhassem no mesmo canvas de grafos narrativos em simultâneo, mimetizando a experiência colaborativa de ferramentas modernas de design como o Figma.
 3. **Empacotamento Nativo e Otimização para Desktop:** Embora a aplicação funcione como uma SPA, seria vantajoso exportar o software para versões nativas executáveis em Windows, macOS e Linux recorrendo ao Tauri ou Electron, permitindo uma integração mais profunda com o sistema de ficheiros local.
 4. **Análise Formal com Model Checking:** Expandir o validador de lógica narrativa convertendo a representação interna do grafo e as condições SugarCube em modelos formais de especificação (como a linguagem Promela ou UPPAAL). Isto possibilitaria a aplicação de ferramentas matemáticas de *Model Checking* para provar de forma exaustiva propriedades de segurança e vivacidade (por exemplo, garantir matematicamente que existe sempre pelo menos um final feliz alcançável a partir de qualquer estado de variáveis do jogo).
 5. **Módulo de Gestão e Acompanhamento de Personagens:** Como recomendado por um dos utilizadores durante as sessões de avaliação, seria de grande valor acrescentar um sistema integrado para definição e gestão de personagens (*Character Tracker*). Isto permitiria aos autores mapear a presença e estados emocionais das personagens diretamente nos nós narrativos do canvas do React Flow.
 6. **Filtragem e Bloqueio de Exportação de Nós Secretos:** Embora a plataforma já permita classificar passagens como “secretas” ou privadas (etiquetando-as de forma visual), sugere-se como trabalho futuro a implementação de um filtro de exportação ativo. Este mecanismo permitiria bloquear a inclusão ou o streaming destas cenas sinalizadas no ficheiro final compilado (Twee/HTML), prevenindo a revelação inadvertida de segredos da trama (*spoilers*) ou a publicação de passagens que estejam em fase de desenvolvimento (*Work in Progress*).

Em suma, o desenvolvimento do *Plot in a Pot* alcançou com sucesso os objetivos propostos, aproximando a engenharia de software e a teoria de grafos da escrita criativa e do *game design*. Ao unificar a robustez da validação de espaço de estados com a flexibilidade da interação visual e acessibilidade inclusiva, a plataforma capacita autores na materialização de narrativas interativas complexas, proporcionando uma solução

integrada que facilita a escrita de histórias não-lineares e serve como base sólida para futuras investigações e desenvolvimentos no domínio dos jogos interativos.

Bibliography

- Aarseth, Espen J. (1997). *Cybertext: Perspectives on Ergodic Literature*. Johns Hopkins University Press.
- Adams, Ernest (2014). *Fundamentals of Game Design*. 3rd. New Riders.
- Articy Software (2010). *articy:draft: Tool for storytelling and narrative design*. URL: <https://www.articy.com/>.
- Babich, Nick (2023). «Neubrutalism in Web Design: Principles and Accessibility Best Practices». Em: *UX Booth*. URL: <https://www.uxbooth.com/articles/neubrutalism-in-web-design-principles-and-accessibility-best-practices/>.
- Bondy, J. A. e U. S. R. Murty (2008). *Graph Theory*. Vol. 244. Graduate Texts in Mathematics. Springer. DOI: 10.1007/978-1-84628-970-5.
- Borges, Jorge Luis (1941). «El jardín de senderos que se bifurcan». Em: *Sur*.
- Brooke, John (1996). «SUS: A 'quick and dirty' usability scale». Em: *Usability evaluation in industry*. Taylor & Francis, pp. 189–194.
- Catmull, Edwin e Raphael Rom (1974). «A class of local interpolating splines». Em: *Computer Aided Geometric Design*. Ed. por Robert E. Barnhill e Richard F. Riesenfeld. Academic Press, pp. 317–326.
- Chen, H. et al. (2021). «GraphPlan: Story generation by planning with event grap». arXiv preprint arXiv:2102.02977.
- Choice of Games (2026). *ChoiceScript License and Terms of Use*. URL: <https://www.choiceofgames.com/make-your-own-games/choicescript-intro/>.
- Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3rd. MIT Press.
- Ferreira, Fábio, Hudson Silva Borges e Marco Tulio Valente (2024). «Refactoring React-based Web apps». Em: *Journal of Systems and Software* 215, p. 112105. DOI: 10.1016/j.jss.2024.112105. URL: <https://doi.org/10.1016/j.jss.2024.112105>.
- Inkle Studios (2016). *Ink: inkle's open source scripting language for writing interactive narrative*. URL: <https://www.inklestudios.com/ink/>.

Interactive Fiction Technology Foundation (2020). *Twee 3 Specification*. URL: <https://github.com/iftechfoundation/twine-specs/blob/master/twee-3-specification.md>.

Interactive Fiction Technology Foundation (2026). *IFTF at GDC 2026: Recap and Reflections*. URL: <https://iftechfoundation.org/news/iftf-gdc-2026/>.

Juul, Jesper (2005). *Half-Real: Video Games between Real Rules and Fictional Worlds*. MIT Press.

Klimas, Chris (2009). *Twine: An open-source tool for telling interactive, nonlinear stories*. URL: <https://twinery.org/>.

Li, Z., X. Ding e T. Liu (2018). «Constructing Narrative Event Evolutionary Graph for Script Event Prediction». arXiv preprint arXiv:1805.05081.

Loranger, Page (2022). *Neobrutarism: The Web Design Trend Challenging Usability Conventions*. Nielsen Norman Group. URL: <https://www.nngroup.com/articles/neobrutarism/>.

Martins, Ana Isabel et al. (2015). «European Portuguese Validation of the System Usability Scale (SUS)». Em: *Procedia Computer Science* 67, pp. 293–300. DOI: 10.1016/j.procs.2015.09.273. URL: <https://doi.org/10.1016/j.procs.2015.09.273>.

Meta Open Source (2013). *React: A JavaScript library for building user interfaces*. URL: <https://react.dev/>.

Montfort, Nick (2005). *Twisty Little Passages: An Approach to Interactive Fiction*. Cambridge, MA: MIT Press.

Mormoris, V. (2024). *Comparing tools for nonlinear storytelling in video games*.

NarrativeFlow (2023). *NarrativeFlow: Visual scripting for interactive stories*. URL: <https://narrativeflow.dev/>.

Ollama Contributors (2024). *Issue #14571: No way to stop control characters output from 'ollama run'*. GitHub Repository Issue Tracker. URL: <https://github.com/ollama/ollama/issues/14571>.

Ollama Contributors (2026). *Pull Request #17112: progress: suppress ANSI control characters when stderr is not a TTY*. GitHub Repository Pull Requests. URL: <https://github.com/ollama/ollama/pull/17112>.

Packer, Edward (1984). *How to Write a Choose Your Own Adventure Book*. Bantam Books.

Secret Lab (2015). *Yarn Spinner: The friendly tool for game dialogue*. URL: <https://yarnspinner.dev/>.

Tailwind Labs (2017). *Tailwind CSS: A utility-first CSS framework for rapid UI development*. URL: <https://tailwindcss.com/>.

Tang, C. et al. (2022). «NGEP: A Graph-based Event Planning Framework for Story Generation». arXiv preprint arXiv:2210.10602.

webkid.io (2021). *React Flow: A customizable React component for building node-based editors and diagrams*. URL: <https://reactflow.dev/>.

Yan, Z. e X. Tang (2023). «Narrative Graph: Telling Evolving Stories Based on Event-centric Temporal Knowledge Graph». Em: *Journal of Systems Science and Systems Engineering*.

Apêndices



Manual de Utilização e Instalação

Este apêndice constitui o Manual de Utilização e Instalação oficial do *Plot in a Pot* (PiaP). Destina-se a autores de narrativas interativas, designers de jogos e programadores que pretendam instalar, configurar e explorar todas as capacidades lógicas, visuais e de tradução da plataforma.

A.1 Requisitos do Sistema e Instalação Local

O PiaP é executado inteiramente do lado do cliente no navegador web (*Single Page Application*), mas requer uma pilha local mínima para desenvolvimento e para a execução do assistente de inteligência artificial local.

A.1.1 Instalação do Editor Web

Para executar o editor localmente, certifique-se de que tem o ambiente de execução Node.js instalado (versão 18 ou superior). Siga os seguintes passos de instalação:

1. Descomprima os ficheiros do projeto numa diretoria de trabalho.
2. Abra um terminal de comandos (cmd ou PowerShell no Windows, bash no macOS/Linux) na diretoria raiz do projeto.
3. Execute o comando de instalação de dependências:

```
npm install
```

Este comando irá descarregar e configurar todas as bibliotecas necessárias, incluindo o React 19, React Flow 11 e Tailwind CSS 3.

4. Após a conclusão bem-sucedida da instalação, inicie o servidor de desenvolvimento local através do comando:

```
npm start
```

A aplicação será inicializada e ficará disponível no seu navegador padrão no endereço `http://localhost:3000`.

A.1.2 Configuração do Servidor de IA Local (Ollama)

A funcionalidade de conversão e importação automática de histórias escritas em linguagem natural depende de um servidor local de inferência de Modelos de Linguagem de Grande Escala (LLMs). O PiaP suporta nativamente o *Ollama*. Para o configurar:

1. Descarregue e instale a versão oficial do Ollama a partir de <https://ollama.com/>.
2. Abra o terminal de comandos e execute o descarregamento do modelo recomendado para processamento de texto:

```
ollama pull llama3:8b
```

Pode também optar por modelos mais leves, como o `qwen2.5:7b` ou `gemma2:2b`, dependendo da memória de vídeo (VRAM) disponível no seu computador.

3. Por padrão, o Ollama executa um servidor em segundo plano que escuta na porta local `http://localhost:11434`.
4. No editor do PiaP, clique no botão “Converter” na barra superior. No diálogo que se abre, selecione o fornecedor “Ollama (Local)” e insira o nome do modelo descarregado (e.g., `llama3`) no respetivo campo.

A.2 Manual de Utilização do Editor Gráfico

A interface gráfica de grafos do PiaP segue uma estética neo-brutalista de alto contraste que facilita a distinção dos componentes visuais da narrativa.

A.2.1 O Canvas Interativo

O espaço central do editor representa a estrutura geral da história sob a forma de um grafo. Pode navegar no canvas utilizando as seguintes interações:

- **Deslocação (Pan):** Clique e arraste com o botão esquerdo do rato num espaço vazio do canvas.
- **Zoom:** Utilize a roda do rato para aproximar ou afastar a visualização. Pode também usar o painel do minimapa no canto inferior esquerdo para se situar no grafo.
- **Seleção de Elementos:** Clique com o botão esquerdo sobre qualquer nó de cena ou aresta para abrir as suas propriedades no painel lateral de inspeção.

A.2.2 Criação e Edição de Nós Narrativos (Cenas)

Para adicionar uma nova cena à história:

1. Clique no botão “+ Adicionar Cena” na barra superior de ferramentas (ou use o atalho de teclado **X**).
2. Um novo nó azul com contorno espesso aparecerá no canvas. Clique duas vezes no nó ou selecione-o para abrir o **Inspetor Lateral**.
3. No Inspetor, pode editar:
 - **ID da Cena:** Um identificador único em formato alfanumérico (ex: `cena_taberna`).
 - **Título da Cena:** O nome de identificação visual exibido na barra do nó no canvas (ex: Interior da Taberna).
 - **Conteúdo da Cena:** O texto literário que será lido pelo jogador.
 - **Etiquetas (Tags):** Palavras-chave separadas por vírgula que modificam o comportamento do nó (ex: `css` para estilos, `secreto` para ocultar da exportação).

A.2.3 Criação de Ligações e Arestas de Escolha

As arestas direcionadas do grafo representam as decisões disponíveis para o jogador no final de cada cena narrativa. Para criar uma ligação:

1. Posicione o ponteiro do rato sobre o ponto de saída circular (*handle* direito) do nó de origem.
2. Clique e arraste o cabo de ligação até ao ponto de entrada (*handle* esquerdo) do nó de destino.
3. Uma aresta curva suave será desenhada entre as cenas.
4. O texto da escolha que o jogador irá visualizar é automaticamente derivado das ligações escritas em formato double-bracket no conteúdo do nó de origem (ex: `[[Entrar na taberna|cena_taberna]]`).
5. **Waypoints Interativos:** Se pretender contornar outros nós, clique duas vezes sobre a aresta criada. Um pequeno ponto de controlo amarelo aparecerá. Arraste-o para deformar a curva de forma flexível. Pode adicionar múltiplos waypoints na mesma aresta.

A.2.4 Agrupamento em Zonas (Capítulos)

Para organizar visualmente projetos de escrita literária de grande envergadura:

1. Clique no botão “+ Adicionar Zona” na barra superior. Um retângulo cinzento com contorno pontilhado aparecerá no ecrã.
2. Atribua um nome à Zona no seu cabeçalho (ex: **Capítulo 1**).
3. Arraste nós narrativos existentes para dentro do retângulo da Zona. Estes passarão a ser filhos geométricos da zona.

4. Ao mover ou arrastar a Zona, todos os nós contidos nela deslocar-se-ão de forma solidária.
5. Pode redimensionar a Zona arrastando o indicador de escala no seu canto inferior direito.

A.3 Programação de Lógica Narrativa e Variáveis

O PiaP suporta a declaração de variáveis globais e condicionais lógicas nativas da macro-linguagem SugarCube.

A.3.1 Gestão de Variáveis (Figma-Style Tokens)

Para gerir as variáveis do jogo sem escrever código manualmente:

1. Selecione o nó especial **StoryInit** no canvas (ou use o menu **Sistema / Código** na barra superior) e clique no botão “+ Criar Variável” no painel do Inspetor lateral. Em alternativa, selecione qualquer outra cena e clique no botão “+/- Alterar Valor” no Inspetor.
2. No diálogo do Gestor de Variáveis que se abre, clique em “Criar variável” no topo. Uma nova linha surgirá na tabela.
3. Edite o nome da variável diretamente (o prefixo \$ é gerido automaticamente pelo sistema), selecione o tipo de dados (**Booleano**, **Número** ou **Texto**) e insira o seu valor inicial.

O motor do PiaP injetará de forma autónoma as macros de inicialização correspondentes (como `<<set $tem_chave to false>>`) no corpo do nó especial **StoryInit**.

A.3.2 Edição de Lógica condicional por Código

No conteúdo de qualquer nó de cena, pode inserir macros SugarCube utilizando a sintaxe de parênteses angulares duplos:

- **Modificação de Estado:** Para alterar o valor de uma variável quando o jogador visita o nó, utilize a macro `<<set>>`:

```
<<set $tem_chave = true>>
```

- **Condições de Escolhas:** Para exibir ou ocultar uma escolha com base no estado das variáveis lógicas, envolva a ligação na macro condicional `<<if>>`:

```
<<if $tem_chave>>
[[Abrir a porta trancada|cena_tesouro]]
<<else>>
A porta está trancada.
<</if>>
```

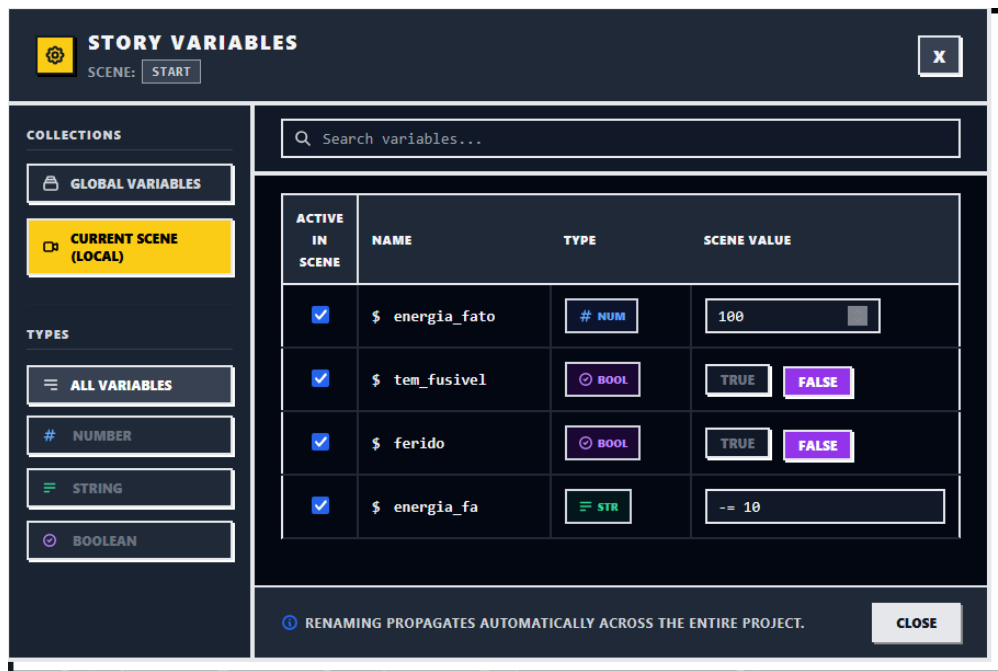


Figura A.1: *Diálogo de gestão de variáveis globais (VariablesModal), apresentando os tokens de dados e a renomeação segura baseada em Regex.*

A.3.3 Editor de Lógica Estruturada

Caso não esteja familiarizado com regras de programação textual, selecione um nó de cena, clique no separador “Visual” e de seguida no botão “Editor de Lógica Visual” no Inspetor:

1. Um painel interativo de formulários estruturados será exibido, dividindo a cena em três secções lógicas: narrativa, modificadores de variáveis (*setters*) e escolhas/-caminhos condicionais.
2. Utilize os menus de seleção (dropdowns) para definir quais as variáveis que mudam de valor ao entrar na cena ou quais os requisitos lógicos (ex: se tem a chave) necessários para libertar uma escolha narrativa.
3. Ao salvar ou alterar os campos, o sistema gera de forma transparente as macros SugarCube correspondentes no texto da cena ativa, sem risco de erros de digitação.

A.4 Tradução e Localização Multi-idioma

A gestão de traduções no PiaP baseia-se numa Matriz de Tradução única que centraliza os textos dos nós.

A.4.1 Configurar Idiomas

Clique no botão “Base de Chaves” no painel lateral direito (Motor de Dados) para abrir a Matriz de Localização:

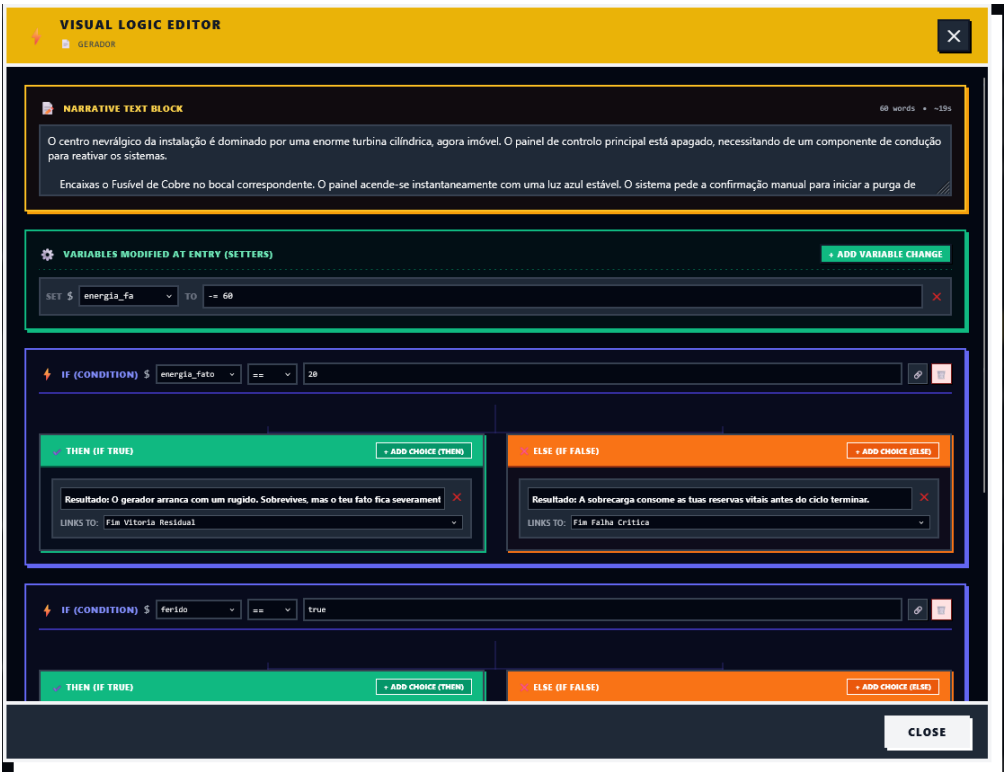


Figura A.2: Interface do editor visual de lógica estruturada (VisualBlocksEditor), dividindo as passagens em painéis de prosa, setters de variáveis e escolhas condicionais.

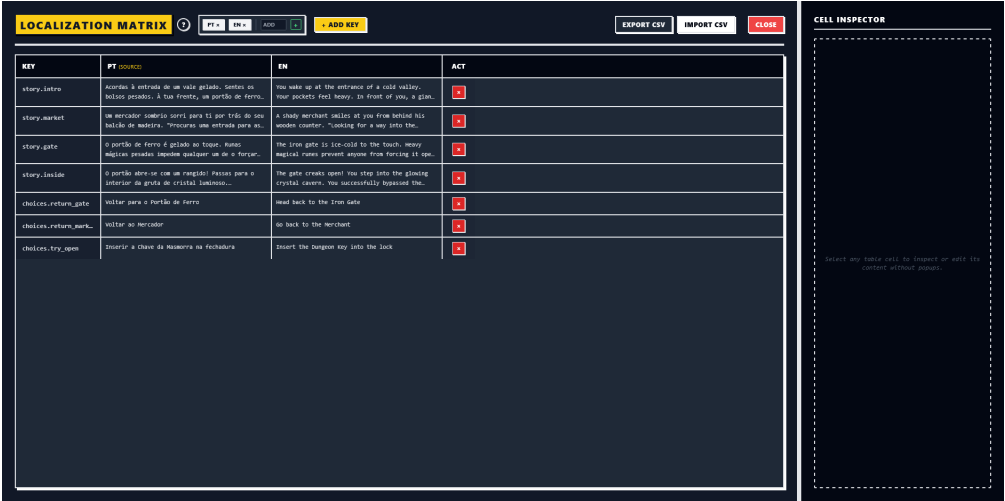


Figura A.3: Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.

1. O idioma nativo padrão é exibido como a primeira coluna.
2. Digite o código de duas ou três letras do novo idioma (ex: **en** para inglês, **es** para espanhol) no campo de texto ao lado do placeholder “ADD” e clique no botão “+”.
3. Uma nova coluna aparecerá na tabela de tradução.

A.4.2 Editar Textos na Matriz

A grelha apresenta todas as cenas narrativas nas linhas e os idiomas nas colunas:

1. Clique duas vezes em qualquer célula correspondente ao idioma secundário.
2. Digite o texto traduzido da cena e clique fora para guardar.
3. Quando a história for executada num idioma secundário, o motor lerá de forma reativa a string traduzida a partir desta matriz.

A.4.3 Importação e Exportação de Ficheiros CSV

Para enviar os ficheiros a um tradutor externo profissional:

1. Clique no botão “Exportar CSV” na Matriz de Tradução. Um ficheiro estruturado em formato tabular será descarregado.
2. O tradutor pode abrir o ficheiro no Microsoft Excel, Google Sheets ou qualquer editor de texto e preencher as colunas dos idiomas secundários.
3. Após a tradução, clique em “Importar CSV” e selecione o ficheiro atualizado. O editor atualizará automaticamente todas as células e strings na memória.

A.5 Validação Lógica e Simulação de Jogo

Antes de disponibilizar o jogo ao público, utilize os motores integrados de auditoria.

A.5.1 O Painel de Validação Estática

O validador estático analisa de forma contínua a correção topológica do grafo. Se existirem falhas lógicas:

- Os nós com erros serão destacados no canvas com uma borda pontilhada a vermelho e um sinal de alerta.
- Clique no botão “Validar Lógica” no painel lateral direito do Motor de Dados para ver a lista estruturada de avisos:
 - **Nós Inacessíveis:** Cenas que nunca podem ser acedidas a partir do nó inicial sob nenhum caminho explorável.
 - **Ligações Partidas (Dead Ends):** Escolhas cujo ID de destino aponta para uma cena inexistente.
 - **Escolhas Bloqueadas:** Hiperligações condicionais cujas expressões lógicas nunca são satisfeitas.

A.5.2 Simulação Ativa (PlayMode) e Testes com o Smart Walker

Clique no botão “Jogar” no painel lateral direito do Motor de Dados para abrir a sandbox de simulação:

1. Jogue a história clicando nas escolhas disponíveis no ecrã de simulação.
2. Utilize o painel de variáveis lateral para ver, em tempo real, as alterações de valores e depurar as macros lógicas.
3. **Smart Walker:** Ative o Smart Walker clicando em “Autoplay”. O simulador iniciará uma exploração automática da história. O Walker utiliza um algoritmo de procura por novidade que prioriza ramos menos visitados. Pode ver o agente a mudar de cenas e a alterar variáveis a alta velocidade no ecrã. Caso o Walker encontre um beco sem saída ou erro lógico, a simulação pausa e destaca o nó problemático.

A.6 Operações de Ficheiros e Publicação do Projeto

O editor suporta leitura e gravação no formato padrão da indústria.

A.6.1 Importar Ficheiros Twee 3

Clique em “Importar ficheiro .twee” na barra de menus e selecione um ficheiro de texto plano em formato Twee 3. O editor processará o cabeçalho e construirá automaticamente o canvas gráfico de nós e arestas correspondente.

A.6.2 Exportar a História

Para guardar o trabalho de autoria ou preparar a narrativa para publicação, o utilizador pode abrir o menu de exportação e descarregar o ficheiro em formato Twee 3 (.twee). São disponibilizados dois modos de exportação:

- **Exportar Base de Dados de Chaves (Keys Database):** Descarrega o ficheiro mantendo as chaves de tradução em formato de tokens (e.g., `t("chave")`). Este modo é ideal para continuar o desenvolvimento noutras sessões e preservar os dados posicionais das passagens e Zonas nos metadados JSON.
- **Exportar História Monolíngue:** Compila a narrativa inteira para um idioma específico (ex: Português, Inglês), resolvendo e injetando as traduções diretamente nas passagens. O ficheiro `.twee` gerado está limpo de chaves técnicas e pronto para ser importado noutros editores ou compilado através do compilador de linha de comandos *Tweego* para gerar a página HTML final do jogo.

B

Guião de Testes e Questionário de Avaliação

Este apêndice apresenta a documentação detalhada utilizada durante as sessões de avaliação com utilizadores realizadas na fase final do projeto. Inclui o guião de tarefas estruturadas fornecido aos participantes e o questionário padronizado *System Usability Scale* (SUS) utilizado para medir a usabilidade percebida.

B.1 Guião de Tarefas de Teste

Os participantes receberam as seguintes instruções para a realização dos testes de usabilidade, sem intervenção ou ajuda dos investigadores:

Tarefa 1: Criação Narrativa e Lógica de Variáveis

Objetivo: Avaliar a facilidade de criação de fluxos e manipulação de estado.

1. Crie cinco nós de cena narrativos no canvas interativo.
2. Defina uma cena inicial e ligue-a a duas cenas intermédias através de opções textuais clássicas do Twine.
3. Num dos nós, insira uma macro de atribuição do SugarCube para criar uma variável de inventário (ex: `<<set $temChave = true>>`).
4. Crie uma aresta condicional para um nó final que apenas fique visível se a condição lógica for satisfeita (ex: `<<if $temChave === true>>`).

Tarefa 2: Importação, Validação e Correção de Erros

Objetivo: Testar a eficácia do motor de validação estática BFS.

1. Importe para a aplicação o ficheiro de teste `story_broken.twee` disponibilizado.
2. Ative o validador lógico e identifique os alertas vermelhos desenhados sobre as arestas e nós.

3. Corrija as ligações quebradas (que apontam para IDs inexistentes) e ligue os nós órfãos identificados pelo validador até que a interface gráfica fique limpa de avisos de erro.

Tarefa 3: Tradução e Localização Multi-idioma

Objetivo: Validar o módulo de localização tabular.

1. Abra a matriz de localização integrada no menu superior da aplicação.
2. Localize as três chaves de cena correspondentes à introdução da história.
3. Introduza a respetiva tradução direta para o idioma Inglês nas respetivas células de grelha.
4. Exporte o dicionário resultante no formato regulamentar CSV.

Tarefa 4: Simulação e Automação com Smart Walker

Objetivo: Testar o PlayMode e a validação por procura de novidade.

1. Inicialize o modo de simulação (*PlayMode*) na barra superior.
2. Jogue a história manualmente durante alguns passos, verificando a reatividade das variáveis de inventário na barra lateral de depuração.
3. Ative o agente autónomo *Smart Walker* e observe a exploração automática do espaço de estados para encontrar caminhos profundos na narrativa.

B.2 Questionário System Usability Scale (SUS)

Após a conclusão das tarefas, cada utilizador preencheu de forma anónima o questionário padronizado SUS, numa adaptação para língua portuguesa das dez perguntas originais do *System Usability Scale* (Brooke, 1996; Martins et al., 2015), disponibilizado em formato digital via Google Forms em https://docs.google.com/forms/d/e/1FAIpQLScRVPQsZ6PEBI_QndGLbjn3Qh3VBzqkTzIlVa29QI3qIsqtCQ/viewform. As respostas foram avaliadas numa escala de Likert de 1 (Discordo Totalmente) a 5 (Concordo Totalmente):

1. Acho que gostaria de utilizar este produto com frequência.
2. Considerei o produto mais complexo do que necessário.
3. Achei o produto fácil de utilizar.
4. Acho que necessitaria de ajuda de um técnico para conseguir utilizar este produto.
5. Considerei que as várias funcionalidades deste produto estavam bem integradas.
6. Achei que este produto tinha muitas inconsistências.
7. Suponho que a maioria das pessoas aprenderia a utilizar rapidamente este produto.
8. Considerei o produto muito complicado de utilizar.
9. Senti-me muito confiante a utilizar este produto.
10. Tive que aprender muito antes de conseguir lidar com este produto.

